Programming in Modern C++

Module M51: C++11 and beyond: General Features: Part 6: Rvalue & Perfect Forwarding

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Weekly Recap

- Introduced several C++11 general features:
  - auto / decltype
  - suffix return type (+ C++14)
  - Initializer List
  - Uniform Initialization
  - Range for Statement
  - constexpr (+ C++14)
  - noexcept
  - nullptr
  - Inline namespace
  - `static_assert`
  - User-defined Literals (+ C++14)
  - Digit Separators and Binary Literals (+ C++14)
  - Raw String Literals
  - Unicode Support
  - Memory Alignment
  - Attributes (+ C++14)
- Understood the difference between Copying & Moving and Lvalue & Rvalue
- Learnt the advantages of Move in C++11 using Rvalue Reference, Move Semantics, and Copy / Move Constructor / Assignment
- Learnt to implement move semantics in UDTs using `std::move` and to implement `std::move`
- Studied a project to code move-enabled UDTs

Module Objectives

- To understand how Rvalue Reference works as a Universal Reference under template type deduction
- To understand the problem of forwarding of parameters under template type deduction
- To learn how Universal Reference and `std::forward` can work for perfect forwarding of parameters under template type deduction
- To understand the implementation of `std::forward`
- To understand how Move is an optimization of Copy

Module Outline

Universal References

Sources:

- Universal References in C++11 – Scott Meyers, isocpp.org, 2012
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses
- Scott Meyers on C++
- Understanding Move Semantics and Perfect Forwarding: Part 1, Part 2: Rvalue References and Move Semantics, Part 3: Perfect Forwarding, Drew Campbell, 2018

Universal References

Reference Collapsing in Templates: Recap (Module 50)

- In C++03, given

```
template<typename T> void f(T& param);
int x;
f<int&>(x);                          // T is int&
```

- `f` is initially instantiated as

```
void f(int& & param);                // reference to reference
```

- C++03's reference-collapsing rule says
  - `T& & => T&`
- So, after reference collapsing, `f`'s instantiation is actually: `void f(int& param);`
- C++11's rules take rvalue references into account:
  - `T& & => T&` `// from C++03`
  - `T&& & => T&` `// new for C++11`
  - `T& && => T&` `// new for C++11`
  - `T&& && => T&&` `// new for C++11`
- Summary:
  - *Reference collapsing involving a* `&` *is always* `T&`
  - *Reference collapsing involving only* `&&` *is* `T&&`

T&& Parameter Deduction in Templates: Recap (Module 50)

- Function templates with a T&& parameter need not generate functions taking a T&& parameter!

```
template<typename T> void f(T&& param); // note non-const rvalue reference
```

- T's deduced type depends on what is passed to param:
  - **Lvalue** ⇒ T is an lvalue reference (T&)
  - **Rvalue** ⇒ T is a non-reference (T)
- In conjunction with reference collapsing:

```
int x;
f(x);              // lvalue: generates f<int&>(int& &&), calls f(int&)
f(10);             // rvalue: generates f<int>(int&&), calls f(int&&)

TVec vt;           // typedef vector<int> TVec;
                   // TVec createTVec();
f(vt);             // lvalue: generates f<TVec&>(TVec& &&), calls f(TVec&)
f(createTVec());   // rvalue: generates f<TVec>(TVec&&), calls f(TVec&&)
```

Universal References

- T&& really is a magical reference type!
  - For lvalue arguments, T&& becomes T& => lvalues can bind
  - For rvalue arguments, T&& remains T&& => rvalues can bind
  - For const/volatile arguments, const/volatile becomes part of T
  - T&& parameters can bind *anything*
- Two conceptual meanings for T&& syntax:
  - **Rvalue reference**. Binds rvalues only

```
void f(Widget&& param);      // takes only non-const rvalue
```

  - **Universal reference**. Binds lvalues and rvalues

```
template<typename T>
void f(T&& param);          // takes lvalue or rvalue, const or non-const
```

    - ▷ Really an rvalues reference in a reference-collapsing context

auto&& ≡ T&&

- `auto` type deduction ≡ template type deduction, so `auto&&` variables are also universal references:

```
int calcVal();
int x;

auto&& v1 = calcVal(); // deduce type from rvalue => v1's type is int&&

auto&& v2 = x;         // deduce type from lvalue => v2's type is int&
```

- Note that `decltype()&&` does not behave like a universal references as it does not use template type deduction:

```
decltype(calcVal()) v3; // deduced type is int

decltype(x) v4;         // deduced type is int
```

Rvalue References vs. Universal References

- Read code carefully to distinguish them
  - Both use && syntax: Occus after a POD or UDT for Rvalue References, but after type variable T for Universal References
  - Type deduction for T for Universal References
  - Behavior is different:
    - Rvalue references bind only rvalues
    - Universal references bind lvalues *and* rvalues
      - that is, may become either T& or T&&, depending on initializer
- Consider `std::vector`:

```
template<class T, class Allocator=allocator<T>> // from C++11 Standard
class vector { public: ...
    void push_back(const T& x);          // lvalue reference
    void push_back(T&& x);               // rvalue reference!
    template<class... Args>
    void emplace_back(Args&&... args);   // universal reference
    ...
};
```

Rvalue Ref. vs. Universal Ref.: Illustration

Post-Recording

```
#include <iostream>
using namespace std;

// overloaded functions for test
void test(const int& a) // lvalue ref
{ cout << "lvalue:" << a << endl; }
void test(int&& a)      // rvalue ref
{ cout << "rvalue: " << a << endl; }

template <typename T>
class Data { T _data; public:
    Data(T data): _data(data)
    { cout << "ctor " << endl; }
    Data(Data&& obj) // move ctor - rvalue ref
    { cout << "mtor " << endl; }
    // class template
    void f1(T&& v) { // rvalue ref
        test(forward<T>(v));
    }
    template<typename U> // member fn. template
    void f2(U&& v) { // universal ref
        test(forward<U>(v)); //
    }
};
```

```
void g(int&& param) // simple fn - rvalue ref
{ test(forward<int>(param)); }

template<typename T>
void f(T&& param)   // template fn - universal ref
{ test(forward<T>(param)); }

int main() { int a = 20;
    //g(a);     // cannot bind rvalue reference of
                // type int&& to lvalue of type int
    g(move(a));     // rvalue: 20
    f(a);           // lvalue: 20
    f(move(a));     // rvalue: 20

    Data<int> d1(10);            // ctor
    Data<int> d2(move(d1)); // mtor: rvalue ref
    //d1.f1(a); // cannot bind rvalue reference of
                // type int&& to lvalue of type int
    d1.f1(move(a)); // rvalue: 20
    d1.f2(a);       // lvalue: 20
    d1.f2(move(a)); // rvalue: 20
}

// For std::forward - See Perfect Forwarding
```

Sources:

- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses
- Scott Meyers on C++
- Perfect Forwarding, modernescpp.com, 2016
- Understanding Move Semantics and Perfect Forwarding: Part 1, Part 2: Rvalue References and Move Semantics, Part 3: Perfect Forwarding, Drew Campbell, 2018

Perfect Forwarding

Perfect Forwarding

- Goal: one function that *does the right thing:*
  - *Copies* lvalue args
  - *Moves* rvalue args
- Solution is a perfect forwarding function:
  - Templatized function forwarding `T&&` params to members
- **What is Perfect Forwarding?**
  - Perfect forwarding allows a template function that accepts a set of arguments to forward these arguments to another function whilst retaining the lvalue or rvalue nature of the original function arguments
- Let us check an example

Perfect Forwarding Example: (broken)

```
#include <iostream>
class Data { int i; public: Data(): i(0) { } }; // a UDT

void g(const int&) { std::cout << "int& in g" << "; "; }  // binds with lvalue parameter
void g(int&&)      { std::cout << "int&& in g" << "; "; } // binds with rvalue parameter

void h(const Data&) { std::cout << "Data& in h" << std::endl; }  // binds with lvalue parameter
void h(Data&&)      { std::cout << "Data&& in h" << std::endl; } // binds with rvalue parameter

template<typename T1, typename T2>
void f(T1&& p1, T2&& p2) { // universal ref. gets lvalue or rvalue from arg by template type deduction
    g(p1); // always binds with lvalue parameter as p1 is an lvalue in f
    h(p2); // always binds with lvalue parameter as p2 is an lvalue in f
}
int main() { int i { 0 }; Data d;
    f(i, d);                  // (lvalue, lvalue) binds with int& in g; Data& in h
    f(std::move(i), d);       // (rvalue, lvalue) binds with int& in g; Data& in h
    f(i, std::move(d));       // (lvalue, rvalue) binds with int& in g; Data& in h
    f(std::move(i), std::move(d)); // (rvalue, rvalue) binds with int& in g; Data& in h
}
```

- Lvalue arg passed to p1 ⇒ g(const int&) receives Lvalue
- Rvalue arg passed to p1 ⇒ g(int&&) receives Lvalue
- Lvalue arg passed to p2 ⇒ h(const Data&) receives Lvalue
- Rvalue arg passed to p2 ⇒ h(Data&&) receives Lvalue

Perfect Forwarding Example: (fixed) by `std::forward`

```
#include <iostream>
class Data { int i; public: Data(): i(0) { } }; // a UDT

void g(const int&) { std::cout << "int& in g" << "; "; }  // binds with lvalue parameter
void g(int&&)      { std::cout << "int&& in g" << "; "; } // binds with rvalue parameter

void h(const Data&) { std::cout << "Data& in h" << std::endl; }  // binds with lvalue parameter
void h(Data&&)      { std::cout << "Data&& in h" << std::endl; } // binds with rvalue parameter

template<typename T1, typename T2>
void f(T1&& p1, T2&& p2) { // universal ref. gets lvalue or rvalue from arg by template type deduction
    g(std::forward<T1>(p1)); // std::forward forwards lvalue arg to lvalue param and
    h(std::forward<T2>(p2)); // rvalue arg to rvalue param
}
int main() { int i { 0 }; Data d;
    f(i, d);                       // (lvalue, lvalue) binds with int& in g; Data& in h
    f(std::move(i), d);            // (rvalue, lvalue) binds with int&& in g; Data& in h
    f(i, std::move(d));            // (lvalue, rvalue) binds with int& in g; Data&& in h
    f(std::move(i), std::move(d)); // (rvalue, rvalue) binds with int&& in g; Data&& in h
}
```

- Lvalue arg passed to `p1` ⇒ `g(const int&)` receives Lvalue
- Rvalue arg passed to `p1` ⇒ `g(int&&)` receives Rvalue
- Lvalue arg passed to `p2` ⇒ `h(const Data&)` receives Lvalue
- Rvalue arg passed to `p2` ⇒ `h(Data&&)` receives Rvalue

Perfect Forwarding

- Despite `T&&` parameters, code fully type-safe:
- Type compatibility verified upon instantiation
  - Only `int`-compatible types valid for call to `g()`
  - Only `Data`-compatible types valid for call to `h()`. For example in the context of

```
...
class DerivedData: public Data { public: DerivedData(): Data() { } };
...
int main() { ... DerivedData d; ... }
```

The code works exactly as before. Whereas for

```
...
class OtherData { int i; public: OtherData(): i(0) { } }; // another UDT
...
int main() { ... OtherData d; ... }
```

The code fails compilation: `error: no matching function for call to h(OtherData&)`

Perfect Forwarding

- The flexibility can be removed via `static_assert` (Module 48) as follows:

```
...
template<typename T1, typename T2>
void f(T1&& p1, T2&& p2) {
    // Asserts that T2 must be of type Data
    static_assert(std::is_same< typename std::decay<T2>::type, Data >::value,
        "T2 must be Data");

    g(std::forward<T1>(p1)); // T1 too may be asserted, if needed
    h(std::forward<T2>(p2));
}
...
class DerivedData: public Data { public: DerivedData(): Data() { } };
...
int main() { ... DerivedData d; ... }
```

Compile-time error: `error: static assertion failed: T2 must be Data`

Perfect Forwarding Example 1: Modified from slide M51.14


```
#include <iostream>
class Data { int i; public: Data(int i = 5): i(i) { }
    operator int() { return i; }                  // cast to int
    Data& operator++() { ++i; return *this; } // pre-increment operator
    Data& operator--() { --i; return *this; } // pre-decrement operator
};
void g(int& a) { std::cout << "int& in g: " << ++a << "; ";  }          // binds non-const lvalue param
void g(int&& a) { std::cout << "int&& in g: " << --a << "; "; }         // binds rvalue param

void h(Data& a) { std::cout << "Data& in h: " << ++a << std::endl; }   // binds non-const lvalue param
void h(Data&& a) { std::cout << "Data&& in h: " << --a << std::endl; } // binds rvalue param

template<typename T1, typename T2>
void f(T1&& p1, T2&& p2) {
    g(...); // called on p1 with or without std::forward
    h(...); // called on p1 with or without std::forward
}
int main() { int i { 0 }; Data d;
    f(i, d);       // (lvalue, lvalue)
    f(5, d);       // (rvalue, lvalue)
    f(i, Data(7)); // (lvalue, rvalue)
    f(5, Data(7)); // (rvalue, rvalue)
}
```

Without std::forward

```
int& in g: 1; Data& in h: 6
int& in g: 6; Data& in h: 7
int& in g: 2; Data& in h: 8
int& in g: 6; Data& in h: 8
```

With std::forward

```
int& in g: 1; Data& in h: 6
int&& in g: 4; Data& in h: 7
int& in g: 2; Data&& in h: 6
int&& in g: 4; Data&& in h: 6
```

Perfect Forwarding Example 2: Generic Factory Method/1

- Let us write a *generic factory method* that should be able to create each arbitrary object. That means that the function should have the following characteristics:
  - Can take an arbitrary number of arguments
  - Can accept lvalues and rvalues as an argument
  - Forwards it arguments identical to the underlying constructor

```
#include <iostream>

template <typename T, typename Arg>  // For efficiency reasons, the function template should
T CreateObject(Arg& a) {             // take its arguments by a non-const lvalue reference
    return T(a);
}
int main() {
    int five=5; // lvalues
    int myFive= CreateObject<int>(five);
    std::cout << "myFive: "  << myFive << std::endl;

    int myFive2= CreateObject<int>(5); // rvalues: error: cannot bind non-const lvalue reference
                                       // of type int& to an rvalue of type int
    std::cout << "myFive2: " << myFive2 << std::endl;
}
```

Perfect Forwarding Example 2: Generic Factory Method/2

- To solve the compilation issues, we can go one of two ways:
  - Change the non-const lvalue reference to a const lvalue reference (that can bind an rvalue). But that is not perfect, because we cannot change the function argument, if needed
  - Overload the function template for a const lvalue reference and a non-const lvalue reference. That is preferred

```
#include <iostream>

template <typename T, typename Arg> T CreateObject(Arg& a) { return T(a); }       // binds lvalues

template <typename T, typename Arg> T CreateObject(const Arg& a) { return T(a); } // binds rvalues

int main() {
    int five = 5; // lvalues
    int myFive = CreateObject<int>(five);
    std::cout << "myFive: "  << myFive << std::endl;  // myFive: 5

    int myFive2 = CreateObject<int>(5); // rvalues
    std::cout << "myFive2: " << myFive2 << std::endl; // myFive2: 5
}
```

Perfect Forwarding Example 2: Generic Factory Method/3

- The solution has two conceptual issues:
  - To support n arguments, we need to overload $2^n + 1$ variations of `CreateObject<T>(...)`. "+1" for the function `CreateObject<T>()` without any argument
  - Without the overload, the forwarding problem would appear for rvalue arguments as they will be *copied* instead of being *moved*
- So we need to use universal reference in `CreateObject<T>(...)` with `std::forward`

```
#include <iostream>

template <typename T, typename Arg>
T CreateObject(Arg&& a) {          // Universal reference
    return T(std::forward<Arg>(a)); // std::forward
}
int main() {
    int five = 5; // lvalues
    int myFive = CreateObject<int>(five);
    std::cout << "myFive: "  << myFive << std::endl;  // myFive: 5

    int myFive2 = CreateObject<int>(5); // rvalues
    std::cout << "myFive2: " << myFive2 << std::endl; // myFive2: 5
}
```

Perfect Forwarding Example 2: Generic Factory Method/4

- `CreateObject<T>()` needs exactly one argument perfectly forwarded to the constructor
- For arbitrary number of arguments, we need a *variadic template* (TBD later)

```
#include <iostream>
#include <string>
#include <utility>
template <typename T, typename ... Args> // Variadic Templates can get an arbitrary number of arguments
    T CreateObject(Args&& ... args) { return T(std::forward<Args>(args)...); }
int main() {
    int five = 5, myFive = CreateObject<int>(five); // lvalues
    std::cout << "myFive: "  << myFive << std::endl;                    // myFive: 5
    std::string str { "Lvalue" }, str2 = CreateObject<std::string>(str);
    std::cout << "str2: " << str2 << std::endl;                         // str2: Lvalue

    int myFive2 = CreateObject<int>(5); // rvalues
    std::cout << "myFive2: " << myFive2 << std::endl;                   // myFive2: 5
    std::string str3 = CreateObject<std::string>(std::string("Rvalue"));
    std::cout << "str3: " << str3 << std::endl;                         // str3: Rvalue
    std::string str4 = CreateObject<std::string>(std::move(str3));
    std::cout << "str4: " << str4 << std::endl;                         // str4: Rvalue

    double doub = CreateObject<double>(); // Arbitrary number of args
    std::cout << "doub: " << doub << std::endl;                         // doub: 0
    struct Data { Data(int i, double d, std::string s) { } } d = CreateObject<Data>(2011, 3.14, str4);
}
```

Perfect Forwarding Example 3: apply Functor/1

- Let us design an `apply` functor that would take a function and its arguments and apply the function on the arguments

```
template<typename F, typename... Ts> // Using variadic template (TBD later)
auto apply(std::ostream& os, F&& func, Ts&&... args)
    -> decltype(func(args...)) { // may not preserves rvalue-ness
    os << "Forwarding:: ";
    return func(args...);         // may not preserves rvalue-ness
}
```

- `args...` are lvalues, but `apply`'s caller may have passed rvalues:
  - Templates can distinguish rvalues from lvalues
  - `apply` might call the wrong overload of `func`

```
class Data { };
Data myData() { return Data(); }

class DataDispatcher { public:
    void operator()(const Data&) { std::cout << "operator()(const Data&) called\n\n"; } // takes lvalue
    void operator()(Data&&) { std::cout << "operator()(Data&&) called\n\n"; }           // takes rvalue
};
int main() { Data d = myData();
    apply(std::cout, DataDispatcher(), d);        // Forwarding:: operator()(const Data&) called
    apply(std::cout, DataDispatcher(), myData()); // Forwarding:: operator()(const Data&) called
                                                  // rvalue forwarded as lvalue!
}
```

Perfect Forwarding Example 3: apply Functor/2

- Naturally, perfect forwarding solves

```
template<typename F, typename... Ts>
auto apply(std::ostream& os, F&& func, Ts&&... args) // return type is same as func's on original args
    -> decltype(func(std::forward<Ts>(args)...)) { // preserves lvalue-ness / rvalue-ness
    os << "Forwarding:: ";
    return func(std::forward<Ts>(args)...);        // preserves lvalue-ness / rvalue-ness
}
...
int main() { Data d = myData();
    apply(std::cout, DataDispatcher(), d);         // Forwarding:: operator()(const Data&) called

    apply(std::cout, DataDispatcher(), myData()); // Forwarding:: operator()(Data&&) called
}
```

- With return type deduction [C++14]

```
template<typename F, typename... Ts>
decltype(auto) apply(std::ostream& os, F&& func, Ts&&... args) { // return type deduction
    os << "Forwarding:: ";
    return func(std::forward<Ts>(args)...);
}
```

Perfect Forwarding Example 3: apply Functor/3

- Perfect forwarding works perfectly with mixed bindings as well

```
#include <iostream>
using namespace std;
class Data { };
Data myData() { return Data(); }

class DataDispatcher { public: // mixed binding for two parameters
    void operator()(const Data&, const Data&){ cout<< "operator()(const Data&, const Data&) called\n\n"; }
    void operator()(const Data&, Data&&){ cout<< "operator()(const Data&, Data&&) called\n\n"; }
    void operator()(Data&&, const Data&){ cout<< "operator()(Data&&, const Data&) called\n\n"; }
    void operator()(Data&&, Data&&){ cout<< "operator()(Data&&, Data&&) called\n\n"; }
};
template<typename F, typename... Ts>
auto apply(ostream& os, F&& func, Ts&&... args) -> decltype(func(forward<Ts>(args)...)) {
    return func(forward<Ts>(args)...);
}
int main() {
    Data d = myData();
    apply(cout, DataDispatcher(), d, d);               // operator()(const Data&, const Data&) called
    apply(cout, DataDispatcher(), d, myData());        // operator()(const Data&, Data&&) called
    apply(cout, DataDispatcher(), myData(), d);        // operator()(Data&&, const Data&) called
    apply(cout, DataDispatcher(), myData(), myData()); // operator()(Data&&, Data&&) called
}
```

std::forward

Sources:

- Universal References in C++11 – Scott Meyers, isocpp.org, 2012
- std::forward, cppreference.com
- Quick Q: What's the difference between std::move and std::forward?, isocpp.org
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses
- Scott Meyers on C++

std::forward

std::forward

- Let us relook at:

```
template<typename T1, typename T2>
void f(T1&& p1, T2&& p2) { ... h(std::forward<T2>(p2)); }
```

  - `T` a reference (that is,`T` is `T&`) ⇒ lvalue was passed to `p2`
    - ▷ `std::forward<T>(p2)` should return lvalue
  - `T` a non-reference (that is, `T` is `T`) ⇒ rvalue was passed to `p2`
    - ▷ `std::forward<T>(p2)` should return rvalue
- `std::forward` is provided in `<utility>` for this
  - Applicable only to *function templates*
  - *Preserves arguments' lvalue-ness / rvalue-ness /* `const`*-ness* when forwarding them to other functions
- Let us take a look at the implementation

std::forward

- C++11 implementations:

```
template<typename T>          // For lvalues (T is T&);
T&&                           // return lvalue reference
forward(typename remove_reference<T>::type& t) noexcept
{ return static_cast<T&&>(t); }

template<typename T>          // For rvalues (T is T);
T&&                           // return rvalue reference
forward(typename remove_reference<T>::type&& t) noexcept
{ return static_cast<T&&>(t); }
```

- By design, param type disables type deduction ⇒ callers must specify T:

```
template<typename T1, typename T2> void f(T1&& p1, T2&& p2)
{ g(std::forward(p1)); ... } // error! Cannot deduce T1 in call to std::forward

template<typename T1, typename T2> void f(T1&& p1, T2&& p2)
{ g(std::forward<T1>(p1)); ... } // fine
```

Move is an Optimization of Copy

Sources:

- Scott Meyers on C++
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses

Move is an Optimization of Copy

Copy Only

- Move requests for copyable types w/o move support yield copies:

```
class MyResource { public: // w/o move support
    MyResource(const MyResource&); // copy ctor

};
class MyClass { public: // with move support
    MyClass(MyClass&& src)  // move ctor
    // request to move r's value
    : w(std::move(src.r)) { ... }
    private: MyResource r; // no move support
};
```

src.r is copied to r:

- std::move(src.r) returns an rvalue of type MyResource
- That rvalue is passed to MyResource's copy constructor

Copy & Move

- If MyResource adds move support:

```
class MyResource { public: // with move support
    MyResource(const MyResource&);     // copy ctor
    MyResource(MyResource&&) noexcept; // move ctor
};
class MyClass { public: // with move support
    MyClass(MyClass&& src) noexcept
    // request to move r's value
    : w(std::move(src.r)) { ... }
    private: MyResource r;
};
```

src.r is moved to r:

- std::move(src.r) returns an rvalue of type MyResource
- That rvalue is passed to MyResource's move constructor via normal overloading resolution

Move is an Optimization of Copy

- Implications:
  - *Giving classes move support can improve performance even for* **move-unaware code**
    - Copy requests for rvalues may silently become moves
  - *Move requests safe for types without explicit move support*
    - Such types perform copies instead
      - For example, all built-in types (POD)
  - *Move support may exist even if copy operations do not*
    - For example, **Move-only types** like `std::thread` and `std::unique_ptr` that are *moveable*, but not *copyable*
  - *Types should support move when moving cheaper than copying*
    - Libraries use moves whenever possible (for example, STL)
- In short:
  - **Give classes move support when moving faster than copying**
  - **Use `std::move` for lvalues that may safely be moved from**

Move is an Optimization of Copy: Use Beyond Construction / Assignment

- Move support useful for other functions, e.g., setters:

```
class MyList { public:
    ...
    void setID(const std::string& newId)          // copy param
    { id = newId; }
    void setID(std::string&& newId) noexcept      // move param
    { id = std::move(newId); }
    void setVals(const std::vector<int>& newVals) // copy param
    { vals = newVals; }
    void setVals(std::vector<int>&& newVals)      // move param
    { vals = std::move(newVals); }
    ...
private:
    std::string id;
    std::vector<int> vals;
};
```

- **Note**:
  - As the move `operator=` of `std::string` is `noexcept`, `setId` is declared `noexcept`
  - Whereas `setVals` is not declared `noexcept`, as the move `operator=` of `std::vector` is not declared `noexcept`

Compiler Generated Move Operations

- Move constructor and move `operator=` are *special*:
  - Generated by compilers under appropriate conditions
- Conditions:
  - *All data members and base classes are movable*
    - Implicit move operations move everything
    - Most types qualify:
      - All built-in types (move ≡ copy).
      - Most standard library types (for example, all containers).
  - Generated operations likely to maintain class invariants
    - *No user-declared copy or move operations*
      - Custom semantics for any ⇒ default semantics inappropriate
      - Move is an optimization of copy
    - *No user-declared destructor*
      - Often indicates presence of implicit class invariant
- More on this later in the Module discussing `default` and `delete`

Compiler Generated Move Operations: Custom Deletion ⇒ Custom Copying


```
class Widget { private:
    std::vector<int> v;
    std::set<double> s;
    std::size_t sizeSum;
public:
    ~Widget() { assert(sizeSum == v.size()+s.size()); }
  ...
};
```

- If `Widget` had implicitly-generated move operations:

```
std::vector<Widget> vw;
Widget w;
...                              // put stuff in w's containers
vw.push_back(std::move(w));      // move w into vw
...                              // no use of w
```

- *User-declared dtor ⇒ no compiler-generated move ops for* `Widget`

Compiler Generated Move Operations: Custom Moving ⇒ Custom Copying

copyable & movable type

```
class Widget1 { private:
    std::u16string name; // copyable/movable type
    long long value;     // copyable/movable type
public: explicit Widget1(std::u16string n);


}; // implicit copy/move ctor
   // implicit copy/move operator=
```

copyable type; not movable

```
class Widget2 { private:
    std::u16string name;
    long long value;
public: explicit Widget2(std::u16string n);
    // user-declared copy ctor
    Widget2(const Widget2& rhs);
}; // => no implicit move ops
   // implicit copy operator=
```

- *Declaring a move operation prevents generation of copy operations*
  - Custom move semantics ⇒ custom copy semantics
    - ▷ **Move is an optimization of copy**

```
class Widget3 { private:                    // movable type; not copyable
    std::u16string name;
    long long value;
public:
    explicit Widget3(std::u16string n);
    Widget3(Widget3&& rhs) noexcept;   // user-declared move ctor => no implicit copy ops
    Widget3&                           // user-declared move op=
    operator=(Widget3&& rhs) noexcept; // => no implicit copy ops
};
```

Module Summary

- Learnt how Rvalue Reference works as a Universal Reference under template type deduction
- Understood the problem of forwarding of parameters under template type deduction and its solution using Universal Reference and `std::forward`
- Learnt the implementation of `std::forward`
- Understood how Move works as an optimization of Copy

Programming in Modern C++

Module M52: C++11 and beyond: General Features: Part 7: Lambda in C++/1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Module Recap

- Learnt how Rvalue Reference works as a Universal Reference under template type deduction
- Understood the problem of forwarding of parameters under template type deduction and its solution using Universal Reference and `std::forward`
- Learnt the implementation of `std::forward`
- Understood how Move works as an optimization of Copy

Module Objectives

- To understand λ expressions (unnamed function objects) in C++
  - Closure Objects
  - Parameters
  - Capture

Module Outline

λ in C++11, C++14, C++17, C++20

Source:

- Lambdas, isocpp.org
- Scott Meyers on C++
- Lambda capture, cppreference.com
- Lambdas: From C++11 to C++20, Part 1 and Lambdas: From C++11 to C++20, Part 2, cppstories.com, 2019
- Lambdas: Smart Pointers, Jim Fix, Reed College

λ in C++11, C++14, C++17, C++20

λ in C++: Closure Object

- A λ *expression* is a mechanism for specifying a *function object* or *functor* (Recall **Module 40**)
- The primary use for a λ is to specify a simple action to be performed by some function
- For example, consider a remainder operation `rem` that computes *m* % *n*, that is, *m* modulo *n*. It has type `int -> int`. To write `rem` in C++, we define a function / functor:

```
int n = 7;
int rem(int m) // Function
    { return m % n; }
// Uses n in context
```

```
int n = 7;
struct remainder {                  // Functor
    int mod;                        // State
    remainder(int n): mod(n) { } // Ctor (n from context)
    int operator()(int m)           // Function call operator
        { return m % mod; }         // Body
};
```

```
rem(23); // 2
```

```
struct remainder rem(n);
rem(23); // 2
```

λ: `auto rem = [n](int m) -> int { return m % n; }` // **Captures** `n` from context

```
rem(23); // 2
```

- Note that `[n]` **Captures** `n` from context to close `rem` and create the **Closure Object** in C++

C++ λ's

- C++11 introduced λ's as syntactically lightweight way to define functions on-the-fly
- λ's can *capture* (or close over) variables from the surrounding scope – by *value* or by *reference*
- First consider callable things that do not capture any variables. C++ offers three alternatives:
  - plain functions (All versions of C & C++)
  - functor classes (C++03 onwards), and
  - lambdas (C++11 onwards)

```
#include <iostream> // cout
using namespace std;

int function (int a) { return a + 3; }
class Functor { public: int operator()(int a) { return a + 3; } };
auto lambda = [] (int a) { return a + 3; };

int main() { Functor functor;
    cout << function(5) << ' ' << functor(5) << ' ' << lambda(5) << endl;
}
8 8 8
```

- For plain functions that capture no variables, lambdas and functors behave the same

C++ λ Syntax and Semantics

- A λ expression consists of the following:

```
[capture list] (parameter list) -> return-type { function body }
```

- The capture list and parameter list can be empty, so the following is a valid λ:

```
[]() { cout << "Hello, world!" << endl; }
```

- *Parameter list* is a sequence of parameter types and variable names as for an ordinary function
- *Function body* is like an ordinary function body
- If the *function body* has only one return statement (which is very common), the *return type* is assumed to be the same as the type of the value being returned
- If there is *no return statement* in the function body, the return type is assumed to be `void`
  - Below λ has return type `void` – can be called without any use of the return value:

    ```
    []() { cout << "Hello from trivial lambda!" << endl; } ();
    ```

  - However, trying to use the return type of the call is an error:

    ```
    cout << []() { cout << "Hello from trivial lambda!" << endl; } () << endl;
    ```

C++ λ Syntax and Semantics

- Below λ returns a `bool` value which is true if the first param is half of the second. The compiler knows the return type as `bool` from the return statement:

```
if ([](int i, int j) { return 2 * i == j; } (12, 24))
    cout << "It's true!";
else
    cout << "It's false!";
```

- To specify return type:

```
cout << "This lambda returns " << [](int x, int y) -> int {
        if(x > 5) return x + y;
        else
            if (y < 2) return x - y; else return x * y;
    } (4, 3) << endl;
```

- Below λ, returns an `int`, though the return statement provides a `double`:

```
cout << "This lambda returns " <<
    [](double x, double y) -> int { return x + y; } (3.14, 2.7) << endl;
```

The output is `"This lambda returns 5"`

C++ λ: Syntax and Semantics

- Below λ *captures* `n` *by value* to compute the value of remainder of `m`:

```
int n = 7;
auto rem = [n](int m) -> int { return m % n; };
```

  - `n` is captured by [n] (value – a copy is made from the context) at the time of constructing the closure object. Hence `n` *must be initialized before the construction* of the closure
  - The *value of* `n` *cannot be changed* within the λ (for immutable λ's)
  - The changes to `n` after the construction of the closure object are not reflected
- Below λ *captures* `s` *by reference* to accumulate the value of `m`:

```
int s = 0;
auto acc = [&s](int m){ s += m; };
```

  - `s` is captured by [&s] (reference – a reference is set to the context) at the time of constructing the closure object. Hence it is *optional to initialize* `s` *before the construction* of the closure. However, it must be initialized before the use of the closure
  - The *value of* `s` *can be changed* within the λ
  - The changes to `s` after the construction of the closure object will be reflected

Lambdas vs. Closures

- **Closure**
  - A *closure* (*lexical* / *function closure*), is a technique for implementing *lexically scoped name binding* in a language with first-class functions
  - Operationally, a closure is a *record storing a function* together with an *environment*
  - The *environment* is a mapping associating (*binding*) each *free* variable of the function with the *value* or *reference* to which the name was bound when the closure was created
  - *Unlike a plain function, a closure allows the function to access captured variables through the closure's copies of their values or references, even in its invocations outside their scope*
- **Lambdas vs. Closures** (From Lambdas vs. Closures by Scott Meyers, 2013)
  - A λ expression `auto f = [&](int x, int y){ return fudgeFactor * (x + y); };` *exists only in a program's source code*. A lambda *does not exist at runtime*
  - The runtime effect of a λ expression is the generation of an object, called *closure*
  - Note that *f* is not the closure, it is a *copy of the closure*. The *actual closure object is a temporary* that's typically destroyed at the end of the statement
  - Each λ expression causes a unique class to be generated (during compilation) and also causes an object of that class type – a closure – to be created (at runtime)
  - Hence, closures *are to* lambdas *as* objects *are to* classes

Closure Objects: Implementing λ's

- A **λ-expression** generates a **Closure Object** at *run-time*
- A closure object is *temporary*
- A closure object is *unnamed*
- For a λ-expression, the compiler creates a *functor class* with:
  - *data members*:
    - a value member each for each value capture
    - a reference member each for each reference capture
  - a *constructor* with the captured variables as parameters
    - a value parameter each for each value capture
    - a reference parameter each for each reference capture
  - a *public inline const function call* `operator()` with the parameters of the lambda as parameters, generated from the body of the lambda
  - *copy constructor*, *copy assignment operator*, and *destructor*
- A *closure object* is constructed as an instance of this class and *behaves like a function object*
- A λ-expression without any capture behaves like a function pointer

Source: C++ Lambda Under the Hood, 2019

Closure Objects: Implementing λ's: Example


```
#include <iostream> //  lambda & closure object
using namespace std;
int main() {
    int val = 0; // for value capture init. must
    int ref;     // for ref. capture init. opt.

    auto check = [val, &ref](int param){
        cout << "val = " << val << ", ";
        cout << "ref = " << ref << ", ";
        cout << "param = " << param << endl;
    };
    // lambda to show captured values
    // constructed with value capture of val
    // and reference capture of ref
    // Also, has a parameter param

    ref = 2;  // init. will be reflected
    check(5); // val = 0, ref = 2, param = 5
    val = 3;  // change will not be reflected
    check(5); // val = 0, ref = 2, param = 5
    ref = 4;  // change will be reflected
    check(5); // val = 0, ref = 4, param = 5
}
```

```
#include <iostream> //  Possible functor by compiler
using namespace std;
int main() {
    int val = 0; // for value capture init. must
    int ref;     // for ref. capture init. opt.

    struct check_f { // functor to show captured values
        int val_f;   // value member for value capture
        int& ref_f;  // ref. member for ref. capture
        check_f(int v, int& r):    // Ctor with
            val_f(v), ref_f(r) { } // value & ref params
        void operator()(int param) const { // param
            cout << "val = " << val_f << ", ";
            cout << "ref = " << ref_f << ", ";
            cout << "param = " << param << endl;
        };
    };
    auto check = check_f(val, ref); // Instantiation

    ref = 2;  // init. will be reflected
    check(5); // val = 0, ref = 2, param = 5
    val = 3;  // change will not be reflected
    check(5); // val = 0, ref = 2, param = 5
    ref = 4;  // change will be reflected
    check(5); // val = 0, ref = 4, param = 5
}
```

Closure Objects: First Class Objects (FCOs)

```
struct trace { int i;
    trace() : i(0) { std::cout << "construct\n"; }
    trace(trace const &) { std::cout << "copy construct\n"; }
    ~trace() { std::cout << "destroy\n"; }
    trace& operator=(trace&) { std::cout << "assign\n"; return *this; }
};
```

Code Snippets	Outputs

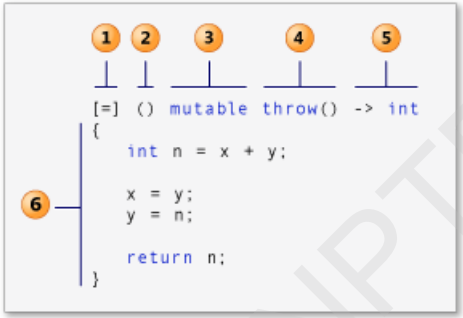
`{ trace t; // t not used so not captured`
`  int i = 8;`
`  auto m1 = [=](){ return i / 2; };`
`}`
	`construct`
	`destroy`

`{ trace t; // capture t by value`
`  auto m1 = [=](){ int i = t.i; };`
`  std::cout << "-- make copy --" << std::endl;`
`  auto m2 = m1;`
`}`
	`construct`
	`copy construct`
	`- make copy -`
	`copy construct`
	`destroy`
	`destroy`
	`destroy`

`{ trace t; // capture t by reference`
`  auto m1 = [&](){ int i = t.i; };`
`  std::cout << "-- make copy --" << std::endl;`
`  auto m2 = m1;`
`}`
	`construct`
	`-- make copy --`
	`destroy`

Closure object has implicitly-declared copy constructor / destructor

Closure Objects: Anatomy



[1] **Capture Clause** (*introducer*)
[2] **Parameter List** (Opt.) (*declarator*)
[3] **Mutable Specs.** (Opt.)
[4] **Exception Specs.** (Opt.)
[5] **(Trailing) Return Type** (Opt.)
[6] **λ body**

λ Expression:: $\mathcal{E} \vdash$ *my_mod* : *Int*, $\lambda(v : Int).\ v \ \%\ my\_mod : Int$
Closure Object:: `[my_mod](int v) -> int { return v % my_mod; }`

- *Introducer*: `[my_mod]`
- *Capture*: `my_mod`
- *Parameters*: `(int v)`
- *Declarator*: `(int v) -> int`
- *Mutable Spec*: Skipped
- *Exception Spec*: Skipped
- *Return Type*: `-> int`
- *λ Body*: `{ return v % my_mod; }`

Closure Objects: Parameters

Parameter Passing	Remarks
`[](){ std::cout << "foo" << std::endl; }();`	`foo`
`[](int v){ std::cout << v << "*6=" << v*6 << std::endl;}(7);`	`7*6=42`

`int i = 7;`
`[](int & v){ v *= 6; } (i);`
`std::cout << "the correct value is: " << i << std::endl;`
	`the correct value is: 42`

`int j = 7;`
`[](int const & v){ v *= 6; } (j);`
`std::cout << "the correct value is: " << j << std::endl;`
	`// error:`
	`// assignment of read-only reference 'v'`

`int j = 7;` `[](int v){ v *= 6; std::cout << "v: " << v << std::endl;}(j);`	`v: 42`

`int j = 7;`
`[](int & v, int j) { v *= j; } (j, 6);`
`std::cout << "j: " << j << std::endl;`
	`// lambda parameters do not affect`
	`// the namespace`
	`j: 42`

`[] std::cout << "foo" << std::endl;  (); is same as` `[]() std::cout << "foo" << std::endl;  ();`	`// lambda expression without a` `// declarator acts as if it were ()`

Closure Objects: Capture

- The *captures* is a comma-separated list of zero or more captures, optionally with default
- The capture list defines the outside variables that are accessible from the λ function body
- The only capture defaults are
  - `[&]` (implicitly capture the used automatic variables *by reference*) and
  - `[=]` (implicitly capture the used automatic variables *by copy / value*)
- The *current object* (`*this`) can be *implicitly captured* if either capture default is present
- If implicitly captured, it is always captured by reference, even for `[=]`. Deprecated since C++20

Capture	Meaning	C++
`identifier`	simple by-copy capture	C++11
`identifier ...`	simple by-copy capture that is a pack expansion	C++11
`identifier init`	by-copy capture with an initializer	C++14
`& identifier`	simple by-reference capture	C++11
`& identifier ...`	simple by-reference capture that is a pack expansion	C++11
`& identifier init`	by-reference capture with an initializer	C++14
`this`	simple by-reference capture of the current object	C++11
`*this`	simple by-copy capture of the current object	C++17
`... identifier init`	by-copy capture with an initializer that is a pack expansion	C++20
`& ... identifier init`	by-reference capture with an initializer that is a pack expansion	C++20

Source: Lambda capture, cppreference.com

Closure Objects: Capture

- Optional captures of λ expressions are (C++11):
  - *Default all by reference*

    ```
    [&](){ ... }
    ```
  - *Default all by value*

    ```
    [=](){ ... }
    ```
  - *List of specific identifier(s) by value or reference and/or this*

    ```
    [identifier](){ ... }
    [&identifier](){ ... }
    [foo,&bar,gorp](){ ... }
    ```
  - *Default and specific identifiers and/or this*

    ```
    [&,identifier](){ ... }
    [=,&identifier](){ ... }
    ```

Source: Lambda capture, cppreference.com

Closure Objects: Capture: Simple Examples


```
int x = 2, y = 3; // Global Context
const auto l0 = []()  { return 1; };        // No capture
typedef int (*l1) (int);                    // Function pointer
const l1 f = [](int i){ return i; };        // Converts to a func. ptr. w/o capture
const auto l2 = [=]() { return x; };        // All by value (copy)
const auto l3 = [&]() { return y; };        // All by ref
const auto l4 = [x]() { return x; };        // Only x by value (copy)
const auto lx = [=x](){ return x; };        // wrong syntax, no need for
                                            // = to copy x explicitly
const auto l5 = [&y]()   { return y; };     // Only y by ref
const auto l6 = [x, &y](){ return x * y; }; // x by value and y by ref
const auto l7 = [=, &x](){ return x + y; }; // All by value except x
                                            // which is by ref
const auto l8 = [&, y]() { return x - y; }; // All by ref except y which
                                            // is by value
const auto l9 = [this]() { }                // capture this pointer
const auto la = [*this](){ }                // capture a copy of *this
                                            // since C++17
```

[&]()->rt{...}: Capture

- *Capture default all by reference*

```
int total_elements = 1;
for_each(cardinal.begin(), cardinal.end(),
    [&](int i) { total_elements *= i; } ); // total_elements
                                           // can be changed
```

- **Errors**

```
[=](int i) { total_elements *= i; } );

error C3491: 'total_elements': a by-value capture cannot be modified
in a non-mutable lambda

[](int i) { total_elements *= i; } );

error C3493: 'total_elements' cannot be implicitly captured because
no default capture mode has been specified
```

[&]()->rt{...}: Capture: Scope & Lifetime

Wrong Capture by Reference

- Closures may outlive their creating function

```
std::function<bool(int)>
    returnClosure(int a) { // returns bool
    int b, c;
    ...
    // won't compile, but assume it would
    return [](int x)
        { return a*x*x + b*x + c == 0; };
}

// f is essentially a copy of
// lambda's closure
auto f = returnClosure(10);
...
if (f(22))  // invoke the closure
```

- What are the values of a, b, c in the call?
  - returnClosure no longer active!
- Non-static locals referenceable only if **captured**

Correct Capture by Reference

- This version has no such problem

```
int a; // now at global or namespace scope
std::function<bool(int)>
    returnClosure() {
    static int b, c; // now static ...

    // now compiles
    return [](int x)
        { return a*x*x + b*x + c == 0; };
}

// as before

auto f = returnClosure();
...
if (f(22))   // as before
```

- a, b, c outlive returnClosure's invocation
- Variables of static storage duration always referenceable

[&]()->rt{...}: Capture


```
// #include <iostream>, <algorithm>, <vector>
template< typename T >
void fill(std::vector<int>& v, T done) { int i = 0; while (!done()) { v.push_back(i++); } }
int main() {
    std::vector<int> stuff; // Fill the vector with 0, 1, 2, ... 7
    fill(stuff, [&]{ return stuff.size() >= 8; }); // [=] compiles but is infinite loop
    for(auto it = stuff.begin(); it != stuff.end(); ++it) std::cout << *it << '  ';
    std::cout << std::endl;

    std::vector<int> myvec; // Fill the vector with 0, 1, 2, ... till the sum exceeds 10
    fill(myvec, [&]{ int sum = 0; // [=] compiles but is infinite loop
        std::for_each(myvec.begin(), myvec.end(), [&](int i){ sum += i; });
                                  // [=] is error: assignment of read-only variable ‘sum’
        return sum >= 10;
    }
    );
    for(auto it = myvec.begin(); it != myvec.end(); ++it) std::cout << *it << '  ';
    std::cout << std::endl;
}
0 1 2 3 4 5 6 7
0 1 2 3 4
```

[=]()->rt{...}: Capture

- *Capture default all by value*

```
std::vector<int> in, out(10);
for (int i = 0; i < 10; ++i)
    in.push_back(i);

int my_mod = 3;
std::transform(in.begin(), in.end(), out.begin(),
               [=](int v) { return v % my_mod; });

for (auto it = out.begin(); it != out.end(); ++it)
    std::cout << *it << ' ';
std::cout << std::endl;
```

```
0 1 2 0 1 2 0 1 2 0
```

[=]()->rt{...}: Capture: Mutable

- Consider

```
int h = 10;
auto two_h = [=] () { h *= 2; return h; };
std::cout << "2h:" << two_h() << " h:" << h << std::endl;

error C3491: 'h': a by-value capture cannot be modified in a non-mutable lambda
```

- λ closure objects have a *public inline function call operator* that:
  - Matches the parameters of the lambda expression
  - Matches the return type of the lambda expression
  - Is declared const
- *Make mutable*

```
int h = 10;
auto two_h = [=] ()  mutable { h *= 2; return h; };
std::cout << "2h:" << two_h() << " h:" << h << std::endl;

2h:20 h:10
```

[=]()->rt{...}: Capture: Mutable


```
int h = 10;
auto f = [=] () mutable { h *= 2; return h; }; // h changes locally
std::cout << "2h:" << f() << std::endl;
std::cout << " h:" << h << std::endl;
```

```
2h:20
 h:10
```

```
int h = 10;
auto g = [&] () { h *= 2; return h; }; // h changes globally
std::cout << "2h:" << g() << std::endl;
std::cout << " h:" << h << std::endl;
```

```
2h:20
 h:20
```

[=]()->rt{...}: Capture: Mutable


```
int i = 1, j = 2, k = 3; // Global i, j, k
auto f = [i, &j, &k]() mutable
{
    auto m = [&i, j, &k]() mutable
    {
        i = 4; // Local i of f
        j = 5; // Local j of m
        k = 6; // Global k
    };
    m();
    std::cout << i << j << k; // Local i of f, Global j, Global k
};
f();
std::cout << " : " << i << j << k; // Global i, j, k
```


426 : 126

[=]()->rt{...}: Capture: Mutable

- Will this compile? If so, what is the result?

```
struct foo {
    foo() : i(0) { }
    void amazing(){ [=]{ i = 8; }(); } // i is captured by value
                                       // Can it be changed without mutable?
    int i;
};
foo f;
f.amazing();
std::cout << "f.i : " << f.i;

Output: f.i : 8
```

- **this** implicitly captured
- `i` actually is `this->i` which can be written from a member function as a data member. So no **mutable** is required

Capture: Restrictions

- Capture restrictions
  - Identifiers must only be listed once

```
[i,j,&z](){...} // Okay
[&a,b](){...}   // Okay
[z,&i,z](){...} // Bad, z listed twice
```

  - Default by value, explicit identifiers by reference

```
[=,&j,&z](){...} // Okay
[=,this](){...}  // Bad, no this with default =
[=,&i,z](){...}  // Bad, z by value
```

  - Default by reference, explicit identifiers by value

```
[&,j,z](){...}  // Okay
[&,this](){...} // Okay
[&,i,&z](){...} // Bad, z by reference
```

- Scope of Capture
  - Captured entity must be defined or captured in the immediate enclosing lambda expression or function

Closure Object: Capture: Mixed Examples


```
class A { std::vector<int> values; int m_;
public: A(int mod) : m_(mod) { }
    A& put(int v) { values.push_back(v); return *this; }
    int extras() { int count = 0;
        std::for_each(values.begin(), values.end(),
            [=, &count](int v){ count += v % m_; });
        return count;
    }
};
A g(4);
g.put(3).put(7).put(8);
std::cout << "extras: " << g.extras();
```

```
extras: 6
```

- Capture default by value
- Capture `count` by reference, accumulate, return
- How do we get `m_`?
- Implicit capture of '`this`' by value

Closure Objects: Capture: Mixed Examples

Capture	Remarks

`int i = 8; // Global context for all lambda's`
`{ int j = 2; auto f = [=]{ std::cout << i / j; };`
`  f();`
`}`
	4

`auto f = [=](){ int j = 2; auto m = [=]{ std::cout << i / j; };`
`  m(); };`
`f();`
	4

`auto f = [i](){ int j = 2; auto m = [=]{ std::cout << i / j; };`
`  m(); };`
`f();`
	4

`auto f = [](){ int j = 2; auto m = [=]{ std::cout << i / j; };`
`  m(); };`
`f();`
	`// Error C3493: 'i' cannot be implicitly`
	`// captured because no default capture`
	`// mode has been specified`

`auto f = [=](){ int j = 2; auto m = [&]{ i /= j; }; m();`
`  std::cout << "inner: " << i; };`
`f(); std::cout << " outer: " << i;`
	`// Error C3491: 'i': a by-value capture`
	`// cannot be modified in a non-mutable`
	`// lambda`

`auto f = [i]() mutable { int j = 2;`
`  auto m = [&i, j]() mutable { i /= j; }; m();`
`  std::cout << "inner: " << i; };`
`f(); std::cout << " outer: " << i;`
	inner: 4
	outer: 8

Module Summary

- Understood λ expressions (unnamed function objects) in C++ with
  - Closure Objects
  - Parameters
  - Capture

Programming in Modern C++

Module M53: C++11 and beyond: General Features: Part 8: Lambda in C++/2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Module Recap

- Understood λ expressions (unnamed function objects) in C++ with
  - Closure Objects
  - Parameters
  - Capture

Module Objectives

- To learn different techniques for writing and using λ expressions in C++
- To understand `std::function` for specifying function prototypes
- To understand generic λ support in C++14
- To learn how to implement recursive λ expressions in C++11 and C++14
- To be exposed to several practice examples

Module Outline

λ in C++: Recap

Source:

- Module 52: C++11 and beyond: General Features: Part 7: Lambda in C++/1

λ in C++: Recap (Module 52)

- A *λ expression* is an unnamed function for specifying lightweight functions in C++
- Compiler generates a functor-like class for a λ which at run-time instantiates a *Closure Object*
- A *λ expression* can have zero or more value and / or reference *parameters* used in its body
- Free variables in the body a *λ expression* must be *captured* by value or by reference
- Here is a complete example

```
class A { std::vector<int> values; int m_;
public: A(int mod) : m_(mod) { }
    A& put(int v) { values.push_back(v); return *this; }
    int extras() { int count = 0;
        std::for_each(values.begin(), values.end(), // iterate over values to apply lambda
            [=, &count] // capture default by value, capture count by reference
            (int v) { count += v % m_; }); // v is a param,  this captured implicitly
                                           // by value & m_ used as this->m_ to accumulate
        return count;
    }
};
int main() { A g(4);          // g.m_ = 4
    g.put(3).put(7).put(8); // g.values = { 3, 7, 8 }
    std::cout << "extras: " << g.extras(); // extras: 6: 3%4 + 7%4 + 8%4 = 3+3+0 = 6
}
```

std::function

Source:

- std::function, cppreference.com
- Usage and Syntax of std::function, stackoverflow.com
- std::function, cplusplus.com
- std::function::function, cplusplus.com

std::function

- `std::function` is defined in `<functional>`

```
template <class T> function;      // empty - undefined
template <class Ret, class... Args> class function<Ret(Args...)>;
```

- It is a polymorphic wrapper for function objects applies to all callable elements:
  - Function pointers
  - Member function pointers
  - Functors (including closure objects)

Type	Old School Define	std::function
Free	`int(*callback)(int,int)`	`function< int(int,int) >`
Functor	`object_t callback`	`function< int(int,int) >`
Member	`int (object_t::*callback)(int,int)`	`function< int(int,int) >`

- Function declarator syntax is: `std::function< R ( A1, A2, A3...) > f;`
- The function object can be copied and moved around, and can be used to directly invoke the callable object with the specified call signature
- The function objects can also be in a state with no target callable object, called *empty functions*. Calling them throws a `bad_function_call` exception

std::function: Examples

```
#include <iostream>     // std::cout
#include <functional>   // std::function, std::negate

int half(int x) { return x/2; }                                  // a function
struct third_t { int operator()(int x) { return x/3; } };        // a function object class
struct MyValue { int value; int fifth() { return value/5; } }; // a class with data members

int main () {
    std::function<int(int)> fn1 = half;                          // function
    std::function<int(int)> fn2 = &half;                         // function pointer
    std::function<int(int)> fn3 = third_t();                     // function object
    std::function<int(int)> fn4 = [](int x) { return x/4; };     // lambda expression
    std::function<int(int)> fn5 = std::negate<int>();            // standard function object

    std::cout << "fn1(60): " << fn1(60) << '\n'; // fn1(60): 30
    std::cout << "fn2(60): " << fn2(60) << '\n'; // fn2(60): 30
    std::cout << "fn3(60): " << fn3(60) << '\n'; // fn3(60): 20
    std::cout << "fn4(60): " << fn4(60) << '\n'; // fn4(60): 15
    std::cout << "fn5(60): " << fn5(60) << '\n'; // fn5(60): -60

    std::function<int(MyValue&)> value = &MyValue::value;  // pointer to data member
    std::function<int(MyValue&)> fifth = &MyValue::fifth;  // pointer to member function
    MyValue sixty {60}; std::cout << "value(sixty): " << value(sixty) << '\n'; // value(sixty): 60
                        std::cout << "fifth(sixty): " << fifth(sixty) << '\n'; // fifth(sixty): 12
}
```

std::function: Example: Pipeline


```
#include <iostream>   // std::cout
#include <algorithm>  // std::for_each
#include <vector>     // std::vector
#include <functional> // std::function
struct machine {
    template< typename T >
    void add(T f) { to_do.push_back(f); } // adding a function to to_do pipeline of functions
    int run(int v) {
        std::for_each(to_do.begin(), to_do.end(), // iterate over the vector of pipeline of functions
            [&v](std::function<int(int)> f) {
            v = f(v); // apply the next function and pipeline the result
            });
        return v;
    }
    std::vector< std::function<int(int)> > to_do; // to_do is a vector collection of pipeline functions
};
int foo(int i) { return i + 4; }
int main() { machine m;
    m.add([](int i){ return i * 3; }); // add a lambda as function #1 in pipeline
    m.add(foo);                        // add foo as function #2 in pipeline
    m.add([](int i){ return i / 5; }); // add a lambda as function #3 in pipeline
    std::cout << "run(7) : " << m.run(7) << std::endl;
}
run(7) : 5 // func. #1 on 7 => 7 * 3 = 21. func. #2 on 21 => 21 + 4 = 25. func. #3 on 25 => 25 / 5 = 5
```

Generic λ in C++14

Source:

- Generic lambdas, ISO
- Lambda expressions (since C++11), cppreference.com
- Scott Meyers on C++
- Lambda expressions in C++, Microsoft
- Generalized Lambda Expressions in C++14
- Generic code with generic lambda expression

Generic λ in C++14

Generic / Generalized λ in C++14

- C++11 introduced λ expressions as short inline anonymous functions that can be nested inside other functions and function calls
- C++ 14 buffed up λ expressions by introducing Generic or Generalized λ
- Following λ function returns the sum of two integers

```
[](int a, int b) -> int { return a + b; } // Return type is optional
```

- Whereas we need a different λ to obtain the sum of two floating point values:

```
[](double a, double b) -> double { return a + b; } // Return type is optional
```

- In C++11 we could unify these two λ functions using template parameters:

```
template<typename T>
[](T a, T b) -> T { return a + b } // Return type is optional - compiler may infer
```

- C++ 14 circumvent this by the keyword auto in the input parameters of the λ expression. Thus the compilers can now deduce the type of parameters during compile time:

```
[](auto a, auto b) { return a + b; } // Compiler must infer return type
```

Generic / Generalized λ in C++14


```
#include <iostream>, #include <string>, using namespace std;

// C++11 lambda's - separate lambda for every type. Return type is optional
auto add_i = [](int a, int b) { return a + b; };                  // Compiler may infer return type
auto add_d = [](double a, double b) { return a + b; };            // Compiler may infer return type
auto add_s = [](string a, string b) { return a + b; };            // Compiler may infer return type
// C++11 templatized lambda - one lambda for multiple types. Return type is optional
template<typename T> auto add_t = [](T a, T b) { return a + b; }; // Compiler may infer return type
// C++14 generic lambda. Return type cannot be specified
auto add = [](auto a, auto b) { return a + b; };                  // Compiler must infer return type
int main () {
    // Different name of each lambda for each type: No inference
    cout << add_i(3, 5);           // add_i for int type                                  // 8
    cout << add_d(2.6, 1.3);       // add_d for double type                               // 3.9
    cout << add_s("Good ", "Day"); // add_s for string type converts from const char* // Good Day

    // Same name of the lambda for all types, type must be specified: No inference
    cout << add_t<int>(3, 5);             // add_t<int> for int type
    cout << add_t<double>(2.6, 1.3);      // add_t<double> for double type
    cout << add_t<string>("Good ", "Day"); // add_t<string> for string type converts from const char*

    // Same name of the lambda for all types and no type need to be specified: It is inferred
    cout << add(3, 5);                            // add for int type
    cout << add(2.6, 1.3);                        // add for double type
    cout << add(string("Good "), string("Day")); // add for string type - cannot convert from const char*
}
```

Return Type of Generic λ in C++14

- The return type may be inferred by the compiler from the return expression:

```
auto add = [](auto a, auto b) { return a + b; };
cout << add(2, 3);      // 5   // rt = int
cout << add(4, 3.3);    // 7.3 // rt = double
cout << add(2.6, 2);    // 4.6 // rt = double
cout << add(3.8, 4.5);  // 8.3 // rt = double
```

- The return type may be specified from the parameters:

```
auto add = [](auto a, auto b) -> decltype(a) { return a + b; };
cout << add(2, 3);      // 5   // rt = int
cout << add(4, 3.3);    // 7   // rt = int
cout << add(2.6, 2);    // 4.6 // rt = double
cout << add(3.8, 4.5);  // 8.3 // rt = double

auto add = [](auto a, auto b) -> decltype(b) { return a + b; };
cout << add(2, 3);      // 5   // rt = int
cout << add(4, 3.3);    // 7.3 // rt = double
cout << add(2.6, 2);    // 4   // rt = int
cout << add(3.8, 4.5);  // 8.3 // rt = double
```

- The return type may be explicitly specified:

```
auto add = [](auto a, auto b) -> int { return a + b; };
```

Use of Generic λ in C++14


```
// Sorting integers, floats, strings using a generalized lambda and sort function
// #include <iostream>, #include <string>, #include <vector>, #include <algorithm>, using namespace std;

// Utility Function to print the elements of a collection
void printElements(auto& C) {
    for (auto e : C) cout << e << " ";
    cout << endl;
}
int main() {
    // Declare a generalized lambda and store it in greater. Works for int, double, string, ...
    auto greater = [](auto a, auto b) -> bool { return a > b; };

    vector<int> vi = { 1, 4, 2, 1, 6, 62, 636 };             // Initialize a vector of integers
    vector<double> vd = { 4.62, 161.3, 62.26, 13.4, 235.5 }; // Initialize a vector of doubles
    vector<string> vs = { "Tom", "Harry", "Ram", "Shyam" };  // Initialize a vector of strings

    sort(vi.begin(), vi.end(), greater); // Sort integers
    sort(vd.begin(), vd.end(), greater); // Sort doubles
    sort(vs.begin(), vs.end(), greater); // Sort strings

    printElements(vi); // 636 62 6 4 2 1 1
    printElements(vd); // 235.5 161.3 62.26 13.4 4.62
    printElements(vs); // Tom Shyam Ram Harry
}
```

Capture-less Generic λ to Function Pointers

- A non-generic λ with an empty capture-list can be converted to a function pointer

```
typedef int (*fp) (int);                // Function pointer
const fp f = [](int i){ return i; }; // Converts to a func. ptr. w/o capture
```

- A capture-less generic λ behaves in the same way
- Further, it *can be converted to more than one compatible function pointers*

```
#include <iostream>

void f(void(*fp)(int))    { fp(1); /*...*/ }
void g(void(*fp)(double)) { fp(2.2); /*...*/ }

int main () {
    auto op = [](auto x) { // generic code for x
        std::cout << "x = " << x << std::endl;
    };

    // use 'op' as a generic callback function pointer
    f(op); // x = 1
    g(op); // x = 2.2
}
```

Recursive λ in C++

Source:

- Recursive lambda expressions in C++, geeksforgeeks.org
- Recursive lambda functions in C++11, stackoverflow.com
- std::function, cppreference.com
- Usage and Syntax of std::function, stackoverflow.com
- std::function, cplusplus.com
- std::function::function, cplusplus.com

Recursive λ in C++

Recursive λ in C++11

- A recursive function $CountOnes(n)$ counts the number of 1's in the binary of n:

$$\begin{aligned} CountOnes(n) &= CountOnes((n-1)/2) + 1, n > 0 \ \&\ odd \\ &= CountOnes(n/2), n > 0 \ \&\ even \\ &= 0, n = 0 \end{aligned}$$

  For example, $CountOnes(5) = 2$ ($5 \equiv 101$) and $CountOnes(14) = 3$ ($14 \equiv 1110$)

- Coded as a normal recursive function:

```
#include <iostream>     // std::cout

// recursive function CountOnes
int CountOnes(int n) { return (0 == n)? 0: CountOnes(n/2) + (n % 2); }

int main() {
    auto Print = // no capture needed for CountOnes or std::cout
                 // as they are available as global symbols
        [](int n) { std::cout << "#(" << n << ") = " << CountOnes(n) << "; "; };

    Print(0); Print(1); Print(2); Print(3); Print(1729);
    // #(0) = 0; #(1) = 1; #(2) = 1; #(3) = 2; #(1729) = 5;
}
```

Recursive λ in C++11

- An attempt to code CountOnes as a lambda runs into compilation error:

```
#include <iostream>     // std::cout

int main() {
    auto CountOnes = [] (int n) { // error: use of 'CountOnes' before deduction of 'auto'
        return (0 == n)? 0: CountOnes(n/2) + (n % 2);
    };
    // ...
}
```

- std::function allows to declare the signature of CountOnes before defining it as a λ:

```
#include <iostream>     // std::cout
#include <functional>   // std::function

int main() {
    std::function<int(int)> CountOnes; // signature of CountOnes
    CountOnes = // capture CountOnes as its use is free in the body
        [&CountOnes] (int n) -> int { return (0 == n)? 0: CountOnes(n/2) + (n % 2); };

    auto Print = // capture needed for CountOnes now as it is a local symbol
        [&CountOnes](int n) { std::cout << "#(" << n << ") = " << CountOnes(n) << "; "; };

    Print(0); Print(1); Print(2); Print(3); Print(1729);
    // #(0) = 0; #(1) = 1; #(2) = 1; #(3) = 2; #(1729) = 5;
}
```

Example 1: Co-recursive Functions


```
#include <iostream>
#include <functional>

int main() {
    std::function<int(int)> f1;
    std::function<int(int)> f2 =
        [&](int i) {
            std::cout << i << " ";
            if (i > 5) { return f1(i - 2); } else { return 0; }
        };

    f1 = [&](int i) { std::cout << i << " "; return f2(++i); };

    f1(10);

    return 0;
}
```

```
10 11 9 10 8 9 7 8 6 7 5 6 4 5
```

Example 2: Factorial


```
#include <iostream>
#include <functional>

int main() {
    std::function<int(int)> fact;
    fact =
        [&fact](int n) -> int
        { return (n == 0) ? 1 : (n * fact(n - 1)); };

    std::cout << "factorial(4) : " << fact(4) << std::endl;

    return 0;
}
```

Example 3: Fibonacci


```
#include <iostream>
#include <functional>
using namespace std;

int main() {
    std::function<int(int)> fibo;
    fibo =
        [&fibo](int n)->int
        { return (n == 0) ? 0 :
                 (n == 1) ? 1 :
                 (fibo(n - 1) + fibo(n - 2)); };

    cout << "fibo(8) : " << fibo(8) << endl;

    return 0;
}
```

Generic Recursive λ in C++14

- A λ does not have a named specific type. So a recursive λ expression needs `std::function` wrapper
- The generic λ expression in C++14 allows recursive λ functions without using `std::function`
- Consider the `CountOnes` example again:

```
#include <iostream>     // std::cout

int main() {
    auto CountOnes = // we need to pass CountOnes as a parameter in the lambda
                     // note the use of CountOnes as the first parameter in the call
        [] (auto&& CountOnes, int n) -> int { // C++14
            return (0 == n)? 0: CountOnes(CountOnes, n/2) + (n % 2);
        };

    auto Print = // capture needed for CountOnes now as it is a local symbol
        [&CountOnes] (int n)
            { std::cout << "CountOnes(" << n << ") = " << CountOnes(CountOnes, n) << std::endl; };

    Print(0); Print(1); Print(2); Print(3); Print(1729);
    // #(0) = 0; #(1) = 1; #(2) = 1; #(3) = 2; #(1729) = 5;
}
```

Generic Recursive λ in C++14: Power


```
#include <iostream>  // C++11 solution using std::function. Works for int base
#include <functional>
int main() {
    std::function<int(int,int)> pow; // pow recursive function. std::function used to define type of pow
    pow = [&pow](int base, int exp) { return exp==0 ? 1 : base*pow(base, exp-1); };
    std::cout << pow(2, 10);       // 2^10 = 1024
    std::cout << pow(2.71828, 10); // e^10 = 1024 // 2.71828 is cast to int giving 2 and wrong result
}

#include <iostream>  // C++14 solution without using std::function. Works for any numeric base
int main() {
    auto power = [](auto self, auto base, int exp) -> decltype(base) { // any numeric 'base' type
        return exp==0 ? 1 : base*self(self, base, exp-1);
    };
    // Wrapper of power to avoid passing power as first parameter to the call
    auto pow = [power](auto base, int exp) -> decltype(base) { return power(power, base, exp); };

    std::cout << power(power, 2, 10);       // 2^10 = 1024 // Needs to pass itself as first parameter
    std::cout << power(power, 2.71828, 10); // e^10 = 22026.3

    std::cout << pow(2, 10);       // 2^10 = 1024 // Wrapper provides a clean solution
    std::cout << pow(2.71828, 10); // e^10 = 22026.3
}
```

Generic Recursive λ in C++14: Factorial


```
#include <iostream>  // Factorial lambda in C++11 solution using std::function
#include <functional>
int main () {
    std::function < int (int) > fact;
    fact = [&fact](int n) -> int
        { return (n == 0) ? 1 : (n * fact(n - 1));
    };
    std::cout << "factorial(4) : " << fact(4) << std::endl;
}
```

```
#include <iostream>  // Factorial lambda in C++14 without using std::function
int main () {
    auto factorial = [](auto self, int n) -> int
        { return (n == 0) ? 1 : (n * self(self, n - 1));
    };
    auto fact = [factorial](int n) -> int // Wrapper of factorial to skip passing factorial
        { return factorial(factorial, n);
    };
    std::cout << "factorial(4) : " << fact(4) << std::endl;
}
```

Generic Recursive λ in C++14: Fibonacci


```
#include <iostream>  // Fibonacci lambda in C++11 solution using std::function
#include <iostream>
#include <functional>
int main () {
    std::function < int (int) > fibo;
    fibo = [&fibo](int n) -> int {
        return (n == 0) ? 0 : (n == 1) ? 1 : (fibo(n - 1) + fibo(n - 2));
    };
    std::cout << "fibo(8) : " << fibo(8) << std::endl;
}

#include <iostream>  // Fibonacci lambda in C++14 without using std::function
int main () {
    auto fibonacci = [](auto self, int n) -> int {
        return (n == 0) ? 0 : (n == 1) ? 1 : (self(self, n - 1) + self(self, n - 2));
    };
    auto fibo = [fibonacci](int n) -> int { // Wrapper of fibonacci to skip passing fibonacci
        return fibonacci(fibonacci, n);
    };
    std::cout << "fibo(8) : " << fibo(8) << std::endl;
}
```

Generalized λ Captures

Source:

- Generalized lambda captures, isocpp.org

Generalized λ Captures in C++14

- In C++11, λs could not (easily) capture by move. Generalized λ capture in C++14 solves the problem and also allows to define arbitrary new local variables in the λ object:

```
auto u = make_unique<some_type>(some, parameters); // a unique_ptr is move-only (TBD later)
go.run([u=move(u)] { do_something_with(u); }); // move the unique_ptr into the lambda
```

- We can also rename the variable `u` the same inside the λ

```
go.run([u2=move(u)] { do_something_with(u2); }); // capture as u2
```

- And we can add arbitrary new state to the λ object, because each capture creates a new type-deduced local variable inside the λ:

```
int x = 4;
int z = [&r = x, y = x + 1] { // y is local, set to 5
        r += 2;               // set x to 6; "r is for Renamed Ref"
        return y + 2;         // return 7 to initialize z
    }();                      // invoke lambda
```

Module Summary

- Learnt different techniques without or with `std::function` to write and use non-recursive and recursive λ expressions in C++11 / C++14
- Several practice examples to be tried and tested

Programming in Modern C++

Module M54: C++11 and beyond: Class Features

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Module Recap

- Learnt different techniques without or with `std::function` to write and use non-recursive and recursive λ expressions in C++11 / C++14
- Several practice examples to be tried and tested

Module Objectives

- Introducing class features in C++11:
  - =default and =delete
  - Control of default move and copy
  - Delegating constructors
  - In-class member initializers
  - Inherited constructors
  - Override controls: override & final
  - Explicit conversion operators
- These features enhance OOP, generic programming, readability, type-safety, and performance in C++11

Module Outline

default / delete Functions

Sources:

- =default and =delete, isocpp.org
- C++ Class and Preventing Object Copy, ariya.io, 2015
- C++ Core Guidelines, github.com
  - C.20: If you can avoid defining default operations, do
  - C.21: If you define or =delete any copy, move, or destructor function, define or =delete them all
  - C.22: Make default operations consistent
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses

default / delete **Functions**

=default and =delete

- The idiom of *prohibiting copying* (for C++03 recall **Module 25**) can now be expressed directly:

```
class X { // ...
    X& operator=(const X&) = delete; // Disallow copying
    X(const X&) = delete;
};
```

- Conversely, we can also say *explicitly* that we want *default copy behavior*:

```
class Y { // ...
    Y& operator=(const Y&) = default; // default copy semantics
    Y(const Y&) = default;
};
```

- Explicitly writing out the default by hand is good for *readability*, but it has two *drawbacks*:
  - it sometimes generates *less efficient code* than the compiler-generated default would, and
  - it prevents types from *being considered PODs*
- The `=default` mechanism can be used for any function that has a default
- The `=delete` mechanism can be used for any function like to eliminate an undesired conversion:

```
struct Z { // ...
    Z(long long);      // can initialize with a long long
    Z(long) = delete; // but not anything smaller
};
```

`default` Member Functions

- The *special* member functions are implicitly generated if used:
  - Default constructor
    - Only if no user-declared constructors
  - Destructor
  - Copy operations (copy constructor, copy `operator=`)
    - Only if move operations not user-declared
  - Move operations (move constructor, move `operator=`)
    - Only if copy operations and destructor not user-declared
- Generated versions are:
  - Public
  - Inline
  - Non-explicit
- `default`ed member functions have:
  - *User-specified declarations with the usual compiler-generated implementations*

default Member Functions

- Typical use: *unsuppress* implicitly-generated functions:

```
class Widget { public:
    Widget(const Widget&); // copy ctor prevents implicitly declared
                           // default ctor & move ops
    Widget() = default;    // declare default ctor, use default impl.
    Widget(Widget&&) noexcept = default; // declare move ctor, use default impl.
    ...
};
```

- Or change *normal accessibility*, explicitness, virtualness:

```
class Widget { public:
    virtual ~Widget() = default;                    // declare as virtual
    explicit Widget(const Widget&) = default;       // declare as explicit
private:
    Widget& operator=(Widget&&) noexcept = default; // declare as private
    ...
};
```

delete Functions

- `deleted` functions are defined, but cannot be used
  - Most common application: prevent object copying:

```
class Widget {
    Widget(const Widget&) = delete;                 // declare and
    Widget& operator=(const Widget&) = delete;      // make uncallable
  ...
};
```

- Note that `Widget` is not movable, either
  - *Declaring* copy operations suppresses implicit move operations!
  - It works both ways:

```
class Gadget {
    Gadget(Gadget&&) = delete;                      // these also
    Gadget& operator=(Gadget&&) = delete;           // suppress copy operations
  ...
};
```

delete Functions

- Not limited to member functions
  - Another common application: control argument conversions
    - ▷ `deleted` functions are declared, hence participate in overload resolution:

```
void f(void*);                      // f callable with any ptr type
void f(const char*) = delete;       // f uncallable with [const] char*
auto p1 = new std::list<int>;       // p1 is of type std::list<int>*
extern char *p2;
...
f(p1);                              // fine, calls f(void*)
f(p2);                              // error: f(const char*) unavailable
f("Modern C++");                    // error
f(u"Modern C++");                   // fine (char16_t* != char*)
```

Control of default move and copy

Sources:

- Control of default move and copy, isocpp.org
- The rule of three/five/zero, cppreference.com
- C++ Core Guidelines, Eds. Bjarne Stroustrup and Herb Sutter, 2022
  - C.20: If you can avoid defining default operations, do
  - C.21: If you define or =delete any copy, move, or destructor function, define or =delete them all
  - C.22: Make default operations consistent
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses

Control of default move and copy

Control of default move and copy

- We learnt in **Module 51** that *Move is an Optimization of Copy*. This is specifically true for *classes having resources* (like pointer / vector) that needs to be moved or copied
- This *needs Move and Copy operations* to be appropriately defined
- We also *need a destructor* in such cases for the release of the resources (like pointer)
- Further, the *compiler provide these functions as default* (if not provided and / or deleted by the user) so that the users do not need to write them for every class
- So there can be one of the following options for each of the *five functions in a class*:

  [1] [Un-declared] Do not mention the function in the class - *implicitly default*
  [2] [=default] Mention the function as `=default` - *explicitly default*
  [3] [Declared] Declare the function but not define it - *prohibit use*
  [4] [=deleted] Mention the function as `=deleted` - *prohibit use*
  [5] [Defined] Provide a user-defined implementation of the function - *proper use*

- For [1] & [2] *compiler provides default implementation* and for [5] *user provides the same*
- In total, we have $5 \times 5 = 25$ *scenarios* but only a few of them are *semantically consistent*
- So for a proper semantics, we *need to control move / copy / destruction*:
  - **Compiler follows a set of rules**
  - **User needs to follow a set of guidelines**

Control of default move and copy: Compiler Rules

- **Move Rules**
  - If *any move, copy, or destructor is explicitly specified* (declared, defined, `=default`, or `=delete`) by the user:
    - ▷ *no move is generated by default*
    - ▷ *any declared but un-defined move is a linker error*
    - ▷ *any* `=default` *move is compiler default*
    - ▷ *any* `=delete` *move is compilation error*
    - ▷ *any un-declared move defaults to corresponding copy*
- Comment / `=default` / `=delete` different functions below to understand the rules of move as well as copy. Note that a default function will not stream any string

```
class X { public:
    X() { std::cout << "Ctor "; }
    X(const X&) { std::cout << "C-Ctor "; }
    X& operator=(const X&) { std::cout << "C= "; return *this; }
    X(X&&) { std::cout << "Mtor "; }
    X& operator=(X&&) { std::cout << "M= "; return *this; }
    ~X() { }
};
```

```
int main() {
    X x1;
    X x2 { x1 };
    x1 = x2;
    X x3 { std::move(x1) };
    x2 = std::move(x3);
}
// Ctor C-Ctor C= Mtor M=
```

Control of default move and copy: Compiler Rules

- **Copy Rules**
  - If *any copy or destructor is explicitly specified* (declared, defined, `=default`, or `=delete`) by the user:
    - *any undeclared copy operations are generated by default*
    - this is deprecated in C++11
  - If *any move is explicitly specified* (declared, defined, `=default`, or `=delete`):
    - *no copy is generated by default*
  - **Bad problem due to default copy in C++03 persists in C++11 onward**

```
class X { public: // copyable class
    ~X() { delete p; } // implicit invariant: p owns *p
    //... // no copy ops declared
    private: int *p;
};
int main() {
    X x1;
    X x2(x1);
} // double delete! of p: run-time error: munmap_chunk(): invalid pointer in gcc
```

Control of default move and copy: User Guidelines

- C++ Core Guidelines, 2022 (Bjarne Stroustrup & Herb Sutter) provides 3 guidelines:
  - [**Rule of zero**]: **C.20: If you can avoid defining default operations, do**
    - Classes with custom Dtors, copy/move Ctors or assignment needs exclusive ownership
    - Other classes should not have these custom functions
  - [**Rule of five**]: **C.21: If you define or =delete any copy, move, or destructor function, define or =delete them all**
    - As the presence of a user-defined (or = default or = delete declared) Dtor, copy Ctor or assignment prevents implicit definition of the move Ctor and assignment, any class for which move semantics are desirable, has to declare all five special member functions
    - [**Rule of three**]: If a class requires a user-defined Dtor, a user-defined copy Ctor, or a user-defined copy assignment, it almost certainly requires all three
  - **C.22: Make default operations consistent**
    - Default operations have a matched set and interrelated semantics. It is a surprise
      - if copy/move construction and assignment do logically different things
      - if constructors and destructors do not provide a consistent view of resource mgmt.
      - if copy and move do not reflect the way constructors and destructors work

Delegating Constructors

Sources:

- Delegating constructors, isocpp.org
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses

Delegating Constructors

Multiple constructors with code copies

```
class Base { public:
    explicit Base(int);
    // ...
};
class Widget: public Base { public: // 4 Ctors
    Widget();
    explicit Widget(double fl);
    explicit Widget(int sz);
    Widget(const Widget& w);
private:
    static int calc(); // calculate Base value
    static constexpr double defaultFlex = 1.5;
    const int size;
    long double flex;
};
```

```
Widget::Widget(): // #1
    Base(calc()), size(0), flex(defaultFlex) {
    regObj(this);
}
Widget::Widget(double fl): // #2
    Base(calc()), size(0), flex(fl) {
    regObj(this);
}
Widget::Widget(int sz): // #3
    Base(calc()), size(sz), flex(defaultFlex) {
    regObj(this);
}
Widget::Widget(const Widget& w): Base(w), // #4
    size(w.size), flex(w.flex) { regObj(this); }
```

Refactored for better reuse with delegated constructors

```
class Widget: public Base { public: // delegation happens when one Ctor calls another - code refactored
    Widget(): Widget(defaultFlex) {}                                          // #1: calls #2
    explicit Widget(double fl): Widget(0, fl) {}                              // #2: calls #5
    explicit Widget(int sz): Widget(sz, defaultFlex) {}                       // #3: calls #5
    Widget(const Widget& w): Base(w), size(w.size), flex(w.flex) { regObj(this); }  // #4: same
private: Widget(int sz, double fl): Base(calc()), size(sz), flex(fl) { regObj(this); } // #5: new
    // ... same as above
};
```

Delegating Constructors

- Delegation is independent of constructor characteristics
  - Delegator and delegatee may each be `inline`, `explicit`, `public` / `protected` / `private`, etc.
  - Delegatees can themselves delegate
  - Delegators' code bodies execute when delegatees return:

```
class Data { int i;
    public:
        Data(): Data(0)   { cout << "Data()" << endl; }    // #1: calls delegated #2
        Data(int i): i(i) { cout << "Data(int)" << endl; } // #2
};

int main() {
    Data d;
}

// Data(int)
// Data()
```

In-class Member Initializers

Sources:

- In-class member initializers, isocpp.org
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses

In-class Member Initializers

- In C++03, *only static const members of integral types* can be initialized in-class *only with a constant expression* so that the initialization can be done at *compile-time*:

```
int var = 7;
class X {
    static const int m1 = 7;          // okay
    const int m2 = 7;                 // error: not static
    static int m3 = 7;                // error: not const
    static const int m4 = var;        // error: initializer not constant expression
    static const string m5 = "odd"; // error: not integral type
    // ...
};
```

- In C++11 a non-static data member may be initialized where it is declared in its class. A constructor can then use the initializer when *run-time* initialization is needed:

```
class A { public: //  C++03
    int a;
    A() : a(7) { }
};
```

```
class A { public: //  C++11
    int a = 7;
};
```

In-class Member Initializers

- This is useful for classes with multiple constructors. Often, all constructors use a common initializer for a member:

```
class A { public: int a, b; //  C++03
    A(): a(7), b(5),
         h_algo("MD5"), s("Ctor run") { }
    A(int a_val): a(a_val), b(5),
         h_algo("MD5"), s("Ctor run") { }
    A(D d): a(7), b(g(d)),
         h_algo("MD5"), s("Ctor run") { }
private: HashingFunction h_algo; // Hash algo
         std::string s;          // Tracer
};
```

```
class A { public: int a, b; //  C++11
    A(): a(7), b(5)
         { }
    A(int a_val): a(a_val), b(5)
         { }
    A(D d): a(7), b(g(d))
         { }
private: HashingFunction h_algo{"MD5"}; // Hash algo
         std::string s{"Ctor run"};     // Tracer
};
```

- If a member is initialized by both an in-class initializer and a constructor, only the constructor's initialization is done (it "overrides" the default). So we can simplify further:

```
class A { public: int a = 7, b = 5; // default initializers
    A() { }
    A(int a_val): a(a_val) { } // Ctor initialization overrides
    A(D d): b(g(d)) { }        // Ctor initialization overrides
private: HashingFunction h_algo{"MD5"};  // Hash algo: Crypto. hash to be applied to all A instances
         std::string s{"Ctor run"};      // Tracer: String indicating state in object lifecycle
};
```

Inheriting Constructors

Sources:

- Inherited constructors, isocpp.org
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses

Inheriting Constructors

A member of a base class is not in the same scope as a member of a derived class:

```
struct B { void f(double); };
struct D : B {

    void f(int);
};
B b;   b.f(4.5); // fine
// surprise: calls f(int) with argument 4
D d;   d.f(4.5);
```

We can "lift" a set of overloaded functions from a base class into a derived class:

```
struct B { void f(double); };
struct D : B {
    using B::f;  // bring all f()s from B into scope
    void f(int); // add a new f()
};
B b;   b.f(4.5); // fine
// fine: calls D::f(double) which is B::f(double)
D d;   d.f(4.5);
```

- Stroustrup has said that *"Little more than a historical accident prevents using this to work for a constructor as well as for an ordinary member function"*
- C++11 provides that facility to life a base class constructor into the derived class
- We present an illustrative example for various svenarios

Inheriting Constructors


```
#include <iostream>
#include <string>

class B { public: // Base class
    B()         { std::cout << "B::B() "; }
    B(int)      { std::cout << "B::B(int) "; }
    void f(int) { std::cout << "B::f(int) "; }
};
class D : public B { public: // Derived class
    using B::f; // lift B::f into D's scope -- works in C++03 and C++11
    void f(string)   { std::cout << "D::f(string) "; } // provide a new overload f
    void f(int)      { std::cout << "D::f(int) "; }    // prefer this override f to B::f(int)
    using B::B; // lift B::B into D's scope -- new in C++11 -- Inheriting Constructors
    // causes implicit declaration of D::D() (or D::D(int)), which, if used, calls B::B() (or B::B(int))
    D(const string&) { std::cout << "D::D(string) "; } // provide a new overloaded constructor
    D(int): B(0)     { std::cout << "D::D(int) "; }    // prefer this overloaded constructor to B::B(int)
};
int main() {
    B b(5);      std::cout << std::endl; // B::B(int)
    D d;         std::cout << std::endl; // B::B() // okay due to ctor inheritance if D::D() is undclared
    D d1(2);     std::cout << std::endl; // B::B(int) D::D(int)
    D d2("ppd"); std::cout << std::endl; // B::B() D::D(string)
    b.f(3);      std::cout << std::endl; // B::f(int)
    d1.f(1);     std::cout << std::endl; // D::f(int)
    d2.f("cd");  std::cout << std::endl; // D::f(string)
}
```

Inheriting Constructors


```
class B: B::B(); B::B(int); B::f(int);
class D: using B::f; D::f(string); D::f(int); using B::B; D::D(const string&); D::D(int);
```

Calls	`// using B::B;`
	`// D(const string&);`
	`// D(int): B(0);`
		`// using B::B;`
		`D(const string&);`
		`// D(int): B(0);`
			`using B::B;`
			`D(const string&);`
			`// D(int): B(0);`
				`using B::B;`
				`D(const string&);`
				`D(int): B(0);`

`B b(5);`
`D d;`
`D d1(2);`
`D d2("ppd");`
	okay
	okay
	error: D::D(int)
	error: D::D(const char[4])
	B::B(int) hidden
		okay
		error: D::D()
		error: D::D(int)
		okay
		B::B(int) hidden
			B::B(int)
			B::B()
			B::B(int)
			B::B() D::D(string)
			D exposes B::B's
				B::B(int)
				B::B()
				B::B(int) D::D(int)
				B::B() D::D(string)
				Overloads

Calls	`// using B::f;`
	`// void f(string);`
	`// void f(int);`
		`// using B::f;`
		`void f(string);`
		`// void f(int);`
			`using B::f;`
			`void f(string);`
			`// void f(int);`
				`using B::f;`
				`void f(string);`
				`void f(int);`

`b.f(3);`
`d1.f(1);`
`d2.f("cd");`
	okay: B::f(int)
	okay: B::f(int)
	error: D::f(const char*)
	D inherits B::f(int)
		okay: B::f(int)
		error: D::f(int)
		okay: D::f(string)
		B::f(int) hidden
			B::f(int)
			B::f(int)
			D::f(string)
			D exposes B::f(int)
				B::f(int)
				D::f(int)
				D::f(string)
				Overload + Override

Inheriting Constructors: Member Initialization in Derived Class

- Inheriting constructors into classes with data members risky. Consider a base class `B`:

```
class B { public:
    explicit B(int);
};
```

- Derive a class `D` with data members:

Default Initialization

```
class D: public B {
public:
    using B::B; // Inherits B::B(int)
private:
    std::u16string name;
    int x, y;
};
D d(10); // compiles, but
// d.name is default-initialized, and
// d.x and d.y are uninitialized
```

In-class Initialization

```
class D: public B {
public:
    using B::B; // Inherits B::B(int)
private:
    std::u16string name = "Uninitialized";
    int x = 0, y = 0;
};
D d(10); // d.name == "Uninitialized",
         // d.x == d.y == 0
```

- Use in-class member initialization when inheriting constructor/s

Override Controls: `override` & `final`

Sources:

- Override controls: override, isocpp.org
- Override controls: final, isocpp.org
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses
- Simulating final Class in C++, geeksforgeeks.org
- Virtual, final and override in C++, 2020
- Why make your classes final?, Andrzej's C++ blog, 2012
- In C++, when should I use final in virtual method declaration?, stackexchange.com

Override Controls: `override` & `final`

Override Controls: `override`


```
struct B { // a base class
    virtual void f();
    virtual void g() const; // may be overridden only by const member function
    virtual void h(char);
    void k(); // not virtual
};
```

A function in a derived class overrides a function in a base class by scoping (without annotation):

```
struct D : B {
    void f(); // overrides B::f()
    void g(); // no override w/o const
    virtual void h(char); // overrides B::h()
    void k(); // no override: B::k() is not virtual
};
```

May be confusing and problematic if a compiler does not warn against suspicious code:

```
struct D : B {
    void f() override; // okay: overrides B::f()
    void g() override; // error: wrong type
    virtual void h(char); // overrides B::h(): warning?
    void k() override; // error: B::k() is not virtual
};
```

- Did the user mean to override `B::g()`? (almost certainly yes)
- Did the user mean to override `B::h(char)`? (probably not because of the redundant explicit virtual)
- Did the user mean to override `B::k()`? (probably, but that's not possible)
- Note:
  - *A declaration marked override is only valid if there is a function to override*. The problem with `h()` may not be caught (because it is correct by the language definition) but it is easily diagnosed
  - `override` is only a *contextual keyword*, so you can still use it as an identifier (not recommended)

Override Controls: `final`

- Sometimes, a programmer wants to prevent a `virtual` function from being overridden. This can be achieved by adding the specifier `final`. For example:

```
struct B {
    virtual void f() const final; // do not override
    virtual void g();
};
struct D : B {
    void f() const; // error: D::f attempts to override final B::f
    void g();       // okay
};
```

- **Why should we use final in C++?**
  - If it is performance (inlining) we want or we simply never want to override, it is typically better not to define a function to be `virtual`
  - This is in contrast to Java where all functions are `virtual` and `final` provides better performance
- It should be used sparingly with care because in a way it contradicts the polymorphic design and in C++ there are other ways to circumvent the required issues in a hierarchy
- Note:
  - *The* `final` *keyword applies to member function, but unlike override, it also applies to types*:

    `class X final { /* ... */ };`

    This prevent the type `X` *to be inherited from*
  - `final` is only a *contextual keyword*, so you can still use it as an identifier (not recommended)

explicit Conversion Operators

Sources:

- Explicit conversion operators, isocpp.org
- explicit specifier, cppreference
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses

explicit Conversion Operators


```
#include <iostream>
#include <string>
using namespace std;
struct Y { explicit Y(const string&) { cout << "Y(string)" << ' '; } };
struct X {
    explicit X(int i) { cout << "X(int)" << ' '; }                                  // C++03 & C++11
    explicit operator int() const { cout << "X::operator int()" << ' '; return 0; }  // C++11 only
    explicit operator Y() const { cout << "X::operator Y()" << ' '; return Y("ppd"); } // C++11 only
};
void fx(const X&) { cout << "fx()" << ' '; } // checker function for conversion to X
void fi(int)      { cout << "fi()" << ' '; } // checker function for conversion to int
void fy(const Y&) { cout << "fy()" << ' '; } // checker function for conversion to Y
int main() { int i { 5 }; X x { 1 }; // X(int)
    fx(i);                  // X(int) fx(): error with explicit X::X(int)
    fx(static_cast<X>(i));  // X(int) fx()
    fi(x);                  // X::operator int() fi(): error with explicit X::operator int()
    fi(static_cast<int>(x)); // X::operator int() fi()
    fy(x);                  // X::operator Y() Y(string) fy(): error with explicit X::operator Y()
    fy(static_cast<Y>(x));  // X::operator Y() Y(string) fy()
}
```

- explicit constructors have been available in C++03
- explicit conversion operators are now available in C++11
- This makes conversion more type safe in C++11
- For casting between unrelated types recap **Module 26** & **Module 33**

explicit Behavior Example


```
struct A { // converting ctor & operator
    A(int) { }
    A(int, int) { } // C++11
    operator bool() const
    { return true; }
};
int main() {
    A a1 = 1;       // OK: copy-initialization selects A::A(int)
    A a2(2);        // OK: direct-initialization selects A::A(int)
    A a3 {4, 5};    // OK: direct-list-initialization selects A::A(int, int)
    A a4 = {4, 5}; // OK: copy-list-initialization selects A::A(int, int)
    A a5 = (A)1;    // OK: explicit cast performs static_cast
    if (a1);        // OK: A::operator bool()
    bool na1 = a1; // OK: copy-initialization selects A::operator bool()
    bool na2 = static_cast<bool>(a1); // OK: static_cast performs direct-initialization

//  B b1 = 1;       // error: copy-initialization does not consider B::B(int)
    B b2(2);        // OK: direct-initialization selects B::B(int)
    B b3 {4, 5};    // OK: direct-list-initialization selects B::B(int, int)
//  B b4 = {4, 5}; // error: copy-list-initialization does not consider B::B(int,int)
    B b5 = (B)1;    // OK: explicit cast performs static_cast
    if (b2);        // OK: B::operator bool()
//  bool nb1 = b2; // error: copy-initialization does not consider B::operator bool()
    bool nb2 = static_cast<bool>(b2); // OK: static_cast performs direct-initialization
}
```

```
struct B { // explicit converting ctor & operator
    explicit B(int) { }
    explicit B(int, int) { }
    explicit operator bool() const
    { return true; }
};
```

explicit Conversion Operators: bool

- explicit operator bool functions treated specially
- Implicit use okay when *safe* (that is, in *contextual conversions*):

```
#include <iostream>
using namespace std;

class X { int *ptr; public:
    explicit X(int *ptr = nullptr): ptr(ptr) { }
    explicit operator bool() const { cout << "X::operator bool()" << ' '; return nullptr == ptr; }
};
int main() { X x1; X x2(new int(5));
    // contextual conversions - permitted in spite of explicit
    if (x1) cout << "NULL" << endl;              // X::operator bool() NULL
    cout << (x2? "NULL": "not-NULL") << endl;    // X::operator bool() not-NULL

    // non-contextual conversions - permitted only on absence of explicit
    cout << (x1 == x2? "Equal": "not-Equal") << endl;
    // Without explicit: x1 and x2 are implicitly converted to bool and compared by bool::operator==
        // X::operator bool() X::operator bool() not-Equal
    // With explicit: implicit conversion of x1 and x2 to bool are disallowed and hence operator== fails
        // error: no match for operator== (operand types are X and X)
}
```

Module Summary

- Introducing several class features in C++11 with examples
- Explained how these features enhance OOP, generic programming, readability, type-safety, and performance in C++11

Programming in Modern C++

Module M55: C++11 and beyond: Non-class Types and Template Features

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Module Recap

- Introduced several class features in C++11 with examples
- Explained how these features enhance OOP, generic programming, readability, type-safety, and performance in C++11

Module Objectives

- To introduce several features in C++11 for non-class types and templates
- To familiarize with enum class and fixed width integer
- To familiarize with variadic templates

Module Outline

1. Other (non-class) Types
   - `enum class`
     - Scope
     - Underlying Type
     - Forward-Declaration
   - Integer Types
   - Generalized `unions`
   - Generalized PODs
2. Templates
   - Extern Templates
   - Template aliases
   - Variadic templates
     - Practice Examples
   - Local types as template arguments
   - Right-angle brackets (Nested Template Closer)
   - Variable templates
3. Module Summary

Other (non-class) Types

Sources:

- `enum class`
  - enum class, isocpp.org
  - An Overview of the New C++ (C++11/14), Scott Meyers Training Courses
  - Closer to Perfection: Get to Know C++11 Scoped and Based Enum Types
  - enum to string in modern C++11 / C++14 / C++17 and future C++20, stackoverflow.com
- Integer Types
  - Fixed width integer types, isocpp.org
  - long long – a longer integer, isocpp.org
  - Extended integer types, isocpp.org
- Generalized `unions`
  - Generalized unions, isocpp.org
- Generalized PODs
  - Generalized PODs, isocpp.org

Other (non-class) Types

Other (non-class) Types

- There have been several additions to non-class types in C++11. They include:
  - `enum class`: These solve several problems for `enum` in C++03
  - **Integer Types**: These include:
    - Fixed width integer types (as enhancements to integer types with size that is standard-defined). This comes from C99 feature
    - `long long` – a longer integer of 64 bits
    - Extended precision in integer types
  - **Generalized `unions`**: That allows rules for using `union` members with ctor / dtor / copy ops as enhancement over C++03
  - **Generalized PODs**: That defines rules for enhanced PODs in C++11
- Important features to learn
  - `enum` class
  - Fixed width integer, and
  - `long long`

enum class

- enum classes (also called: *new* enums, *strong* enums, *scoped* enums) address 3 problems with C++03 enumerations:
  - C++03 enums *implicitly convert to an integer*, causing errors when someone does not want an enumeration to act as an integer
  - C++03 enums *export their enumerators to the surrounding scope*, causing name clashes
  - The *underlying type of an* enum *cannot be specified* in C++03, causing confusion, compatibility problems, and makes forward declaration impossible
- enum classes (*strong* enum) are strongly typed and scoped:

```
enum Alert { green, yellow, orange, red }; // C++03 enum
enum class Color { red, blue };    // scoped and strongly typed enum
                                   // no export of enumerator names into enclosing scope
                                   // no implicit conversion to int
enum class TrafficLight { red, yellow, green };
Alert a = 7;              // error (as ever in C++03)
Color c = 7;              // error: no int->Color conversion
int a2 = red;             // okay: Alert->int conversion
int a3 = Alert::red;      // error in C++03; okay in C++11
int a4 = blue;            // error: blue not in scope
int a5 = Color::blue;     // error: not Color->int conversion
Color a6 = Color::blue;   // okay
```

enum class: Scopes

- Consider `enum` in C++03:

```
enum Color { Bronze, Silver, Gold };
enum Bullion { Silver, Gold };
enum Metal { Silver, Gold, Platinum };
enum CreditCard { Silver, Gold, Platinum };
```

- `Silver` and `Gold` clash in names between `Color`, `Bullion`, `Metal` and `CreditCard`. In C++11, we can use scoped enum:

```
enum class Color { Bronze, Silver, Gold };
enum class Bullion { Silver, Gold };
enum class Metal { Silver, Gold, Platinum };
enum class CreditCard { Silver, Gold, Platinum };
```

- No clash of names as enumerators of a scoped `enum` use a qualified name with enclosing scope:

```
Color col1 = Bronze; // error, Bronze not in scope

Color col2 = Color::Bronze;                                  // OKay
if ((col2 == Color::Silver) || (col2 == Color::Gold)) // OKay
//...
```

enum class: Underlying Type

- Specification of underlying type (optional) now permitted provided every value fits the type:

```
enum Color: unsigned int { red, green, blue };
enum Weather: std::uint8_t { sunny, rainy, cloudy, foggy };

enum Status: std::uint8_t { pending, ready, unknown = 9999 }; // error! unknown does not fit size

enum Color { red, green, blue };   // okay -- type specification is optional as in C++03
```

- *Strongly typed* enums:
  - No implicit conversion to int
    - No comparing scoped enums values with ints
    - No comparing scoped enums objects of different types.
    - Explicit cast to int (or types convertible from int) okay
  - Values scoped to enum type
  - Underlying type defaults to int

```
enum class Elevation: char { low, high }; // underlying type = char
enum class Voltage { low, high };         // underlying type = int
Elevation e = low;                        // error! no low in scope
Elevation e = Elevation::low;             // okay
int x = Voltage::high;                    // error! no conversion to int
if (e) ...                                // error! no conversion to bool
if (e == Voltage::high) ...               // error! no conversion from Elevation to Voltage
```

enum class: Forward-Declaration

- enums of *known size* may be *forward-declared*:

```
enum Color;                                  // as in C++03, error!: size unknown
enum Weather: std::uint8_t;                  // okay
enum class Elevation;                        // okay, underlying type implicitly int
double atmosphericPressure(Elevation e);     // okay
```

Fixed Width Integer Types

- Size of integral types in C++03 are implementation-defined:
  - `sizeof(unsigned char)` = 1 byte: This is standard defined
  - `sizeof(char)`, `sizeof(short)`, `sizeof(int)`, etc.: unspecified
  - The following order is only guaranteed:
    `sizeof(unsigned char)` <= `sizeof(char)` <= `sizeof(short)` <= `sizeof(int)` <= `sizeof(long)`
- C++11 provides fixed width integer types in `<cstdint>` for `N` = { `8`, `16`, `32`, `64` }:
  - `int<N>_t` (`uint<N>_t`): For example, `int8_t` (`uint8_t`)
    - ▷ signed (unsigned) integer type with width of exactly `N` bits with no padding bits
    - ▷ signed integer type to use 2's complement for negative values
  - `int_fast<N>_t` (`uint_fast<N>_t`): For example, `int_fast8_t` (`uint_fast8_t`)
    - ▷ fastest signed (unsigned) integer type with width of at least `N` bits
  - `int_least<N>_t` (`uint_least<N>_t`): For example, `int_least8_t` (`uint_least8_t`)
    - ▷ smallest signed (unsigned) integer type with width of at least `N` bits
  - `intmax_t` (`uintmax_t`):
    - ▷ maximum-width signed (unsigned) integer type

Extended Size & Precision of integers

- What is the difference between the `int` types: `int8_t`, `int_least8_t`, and `int_fast8_t`?
  - Suppose we have a C compiler for a 36-bit system, with `sizeof(char)` = 9 bits, `sizeof(short)` = 18 bits, `sizeof(int)` = 36 bits, and `sizeof(long)` = 72 bits. Then
    - `int8_t` does not exist, because there is no way to satisfy the constraint of having *exactly 8 value bits with no padding*
    - `int_least8_t` is a `typedef` of `char`. NOT of `short` or `int`, because the standard requires the *smallest type with at least 8 bits*
    - `int_fast8_t` can be anything. It is likely to be a `typedef` of `int` if the *native* size is considered to be *fast*
- C++11 provides support for `long long` – a longer integer
  - An integer that's at least 64 bits long. For example:

    ```
    long long x = 9223372036854775807LL;
    ```

  - No, there are no `long long long`s nor can `long` be spelled `short long long`
- C++11 provides support for extended integer (precision) types with a set of rules

Generalized unions

- In C++03, a *member with a user-defined ctor, dtor, or assignment cannot be a member of a* `union`:

```
union U {
    int m1;
    complex<double> m2; // error (silly): complex has constructor
    string m3;          // error (not silly): string has an invariant maintained by ctor, copy, & dtor
};
```

- Obviously, it is illegal to write one member and then read another

```
U u;              // which constructor, if any?
u.m1 = 1;         // assign to int member
string s = u.m3; // disaster: read from string member
```

- C++11 allows a member of types with ctor and dtor. It also adds a restriction to make the more flexible unions less error-prone by encouraging the building of discriminated unions
- *Union member types are restricted*:
  - *No virtual functions*, *No references*, *and No bases* (as ever)
  - *If a union has a member with a user-defined ctor, copy, or dtor then that special function is deleted; that is, it cannot be used for an object of the union type*. This is new. For example:

```
union U1 {
    int m1;
    complex<double> m2; // okay
};
```

```
union U2 {
    int m1;
    string m3;  // okay
};
```

  - This may look error-prone, but the new restriction helps

Generalized unions

- Consider:

```
U1 u;             // okay
u.m2 = { 1, 2 }; // okay: assign to the complex member
U2 u2;            // error: the string destructor caused the U2 destructor to be deleted
U2 u3 = u2;       // error: the string copy constructor caused the U2 copy constructor to be deleted
```

- Basically, U2 is useless unless it is in a *discriminated unions*, such as:

```
class Widget { private: // Three alternative implementations represented as a union
    enum class Tag { point, number, text } type;     // discriminant
    union { point p;  /* point has constructor */ int i;
            string s; // string has default ctor, copy operations, and dtor
    }; // ...
    widget& operator=(const widget& w) { // necessary because of the string variant
        if (type==Tag::text && w.type==Tag::text) { s = w.s; // usual string assignment
            return *this;
        }
        if (type==Tag::text) s.~string();        // destroy (explicitly!)
        switch (w.type) {
            case Tag::point: p = w.p; break;     // normal copy
            case Tag::number: i = w.i; break;
            case Tag::text: new(&s)(w.s); break; // placement new
        }
        type = w.type; return *this;
    }
};
```

Generalized PODs

- A POD (*Plain Old Data*) is something that can be manipulated like a C struct, for example, *bitwise copyable* with `memcpy()`, *bitwise initializable* with `memset()`, etc.
- In C++03 a POD is decided by a set of restrictions on the features used in the definition of a `struct`:

```
struct S { int a; };                                          // Is a POD
struct SS { int a; SS(int aa): a(abs(aa)) { assert(a>=0); } }; // Not a POD in C++03; a POD in C++11
struct SSS { virtual void f(); /* ... */ };                   // Definitely not POD
```

- In C++11, `S` and `SS` are *standard layout types* (a superset of *POD types*) where the ctor does not affect the layout (so `memcpy()` would be fine), only the initialization rules do (`memset()` would be bad)
- However, `SSS` will still have the `vptr` and will not be anything like *plain old data*. C++11 defines:
  - POD Types: Check by `is_pod<T>::value` of type `bool` (deprecated in C++20)
  - Trivially Copyable Types: Check by `is_trivially_copyable<T>::value` of type `bool`
  - Trivial Types: Check by `is_trivial<T>::value` of type `bool`, and
  - Standard-Layout Types: Check by `is_standard_layout<T>::value` of type `bool`

  to deal with various technical aspects of what used to be PODs. POD is defined recursively:
  - *If all members and bases are PODs, then it is a POD*
  - *Naturally: No virtual functions, No virtual bases, No references, and No multiple access specifiers*
- In C++11, PODs is that adding or subtracting constructors do not affect layout or performance

Templates

Sources:

- **Extern templates**
  - Extern templates, isocpp.org
- **Template aliases**
  - Template aliases; Type alias, alias template (since C++11), isocpp.org
  - Type alias, alias template (since C++11)
  - Alias Templates and Template Parameters, 2021
- **Variadic templates**
  - Variadic templates, isocpp.org
  - Variadic templates in C++, Eli Bendersky, 2014
- **Local types as template arguments**
  - Local types as template arguments, isocpp.org
- **Right-angle brackets (Nested Template Closer)**
  - Right-angle brackets, isocpp.org
- **Variable templates (C++14)**
  - Variable templates, isocpp.org

Templates

Templates

- There have been several additions to templates in C++11. They include:
  - **Extern templates**: Used to suppress multiple instantiations
  - **Template aliases**: Used to make a template *just like another template*
  - **Variadic templates**: These are templates with variable number of parameters that are useful in various contexts like writing a type-safe `printf` or defining a `tuple`
  - **Local types as template arguments**: Uses for local as well as unnamed types as template arguments
  - **Right-angle brackets (Nested Template Closer)**: Fixes "`>>`" issue of C++03 for nested templates
  - **Variable templates** (C++14): Variables can now be directly templatized
- Important features to learn:
  - Variadic templates, and
  - Nested template closer

Extern Templates

- A template specialization can be explicitly declared as a way to suppress multiple instantiations. For example:

```
#include "MyVector.h"

// Suppresses implicit instantiation below --
// MyVector<int> will be explicitly instantiated elsewhere
extern template class MyVector<int>;

void foo(MyVector<int>& v) {
    // use the vector in here
}
```

- The *elsewhere* might look something like this:

```
#include "MyVector.h"
template class MyVector<int>; // Make MyVector available to clients
                              // For example, of the shared library
```

- This is basically a way of avoiding significant redundant work by the compiler and linker

Template aliases

- We can make a template *like another template* with a few of template arguments bound:

```
template<class T>
using Vec = std::vector<T, My_alloc<T>>;  // standard vector using my allocator
Vec<int> fib = { 1, 2, 3, 5, 8, 13 };     // allocates elements using My_alloc
vector<int, My_alloc<int>> verbose = fib; // verbose and fib are of the same type
```

- `using` is used to get a linear notation where *name is followed by what it refers to*. Also, *we can alias a set of specializations but we cannot specialize an alias*:

```
// int_exact_trait<N>::type is a type with exactly N bits
template<int> struct int_exact_traits  { typedef int type; };
template<> struct int_exact_traits<8>  { typedef char type; };
template<> struct int_exact_traits<16> { typedef char[2] type; };
// ... define alias for convenient notation
template<int N> using int_exact = typename int_exact_traits<N>::type;
int_exact<8> a = 7; // int_exact<8> is an int with 8 bits
```

- Type aliases can also be used as a different syntax for ordinary type aliases:

```
typedef void (*PFD)(double);        // C style
using PF = void (*)(double);        // using plus C-style type
using P = auto (*)(double) -> void; // using plus suffix return type
```

Template aliases: Example: Matrix

- Consider template class `Matrix`:

```
template <typename T, int Line, int Col>
class Matrix { ... };
```

- `Matrix` has 3 parameters. The type parameter `T`, and the non-type parameters `Line`, and `Col`
- For readability, we want to have two special matrices: a `Square` and a `Vector`. A `Square`'s number of lines and columns should be equal. A `Vector`'s line size should be one.

```
template <typename T, int Line>
using Square = Matrix<T, Line, Line>; // #1

template <typename T, int Line>
using Vector = Matrix<T, Line, 1>;    // #2
```

- `using` declares a type alias (#1 & #2). While the primary template `Matrix` can be parametrized in the three dimensions `T`, `Line`, and `Col`, the type aliases `Square` and `Vector` reduce the parametrization to the two dimensions `T` and `Line`
- Template alias creates names for partially bound templates. Using `Square` and `Vector` is easy:

```
Matrix<int, 5, 3> ma;
Square<double, 4> sq; // Matrix<double, 4, 4>
Vector<char, 5> vec;  // Matrix<char, 5, 1>
```

Variadic templates: printf

- Let us start by implementing printf – the most well-known *variadic function*. Consider:

```
const char* pi = "pi";
const char* m = "The value of %s is about %g (unless you live in %s)\n";
printf(m, pi, 3.14159, "Indiana"); // int printf(const char *format, ...) in C
```

- The simplest case of printf() is when there are *no arguments except the format string*:

```
void printf(const char* s) {
    while (s && *s) {
        if (*s=='%' && *++s!='%') // make sure that there was not meant to be more args (%% for %)
            throw std::runtime_error("invalid format: missing arguments"); // from <exception>
        std::cout << *s++;
    }
}
```

- That done, we must handle printf() with more arguments (*recursive*):

```
template<typename T, typename... Args>              // note the "..."
void printf(const char* s, T value, Args... args) { // recursive function. note the "..."
    while (s && *s) {
        if (*s=='%' && *++s!='%') {      // a format specifier (ignore which one it is)
            std::cout << value;          // use first non-format argument
            return printf(++s, args...); // "peel off" first argument: recursive call
        }
        std::cout << *s++;
    }
    throw std::runtime_error("extra arguments provided to printf");
}
```

Variadic templates: printf

- The code *peels off* the first non-format arg. and then calls itself recursively. When there is no more non-format arg., it calls the first `printf()` – *functional programming at compile time*
- The `Args...` defines what is called a *parameter pack* – a sequence of (type/value) pairs to `peel off` arguments starting with the first:
  - When `printf()` is called with one argument, the first `printf(const char*)` is chosen
  - When `printf()` is called with two or more arguments, the second `printf(const char* s, T value, Args... args)` is chosen, with the first argument as `s`, the second as `value`, and the rest (if any) bundled into the parameter pack args for later use
  - In the call `printf(++s, args...)` the parameter pack args is expanded so that the next argument can now be selected as `value`
  - This carries on until `args` is empty so that the first `printf()` is called
- For generic functional programming, we declare and use a simple variadic template function:

```
template<class ... Types>
    void f(Types ... args); // variadic template function. That is, a function that can take
                            // an arbitrary number of arguments of arbitrary types
f();        // OK: args contains no arguments
f(1);       // OK: args contains one argument: int
f(2, 1.0);  // OK: args contains two arguments: int and double
```

Variadic templates: adder

- Let us implement a function that adds all of its arguments together:

```
template<typename T> T adder(T v) { cout << __PRETTY_FUNCTION__ << endl; return v; }

template<typename T, typename... Args> // template parameter pack: typename... Args
T adder(T first, Args... args)         // function parameter pack: Args... args
{ cout << __PRETTY_FUNCTION__ << endl; return first + adder(args...); }
```

- And we could call and trace it as: `long sum = adder(1, 2, 3, 8, 7); // 21`

```
T adder(T, Args ...) [with T = int; Args = int, int, int, int] // __PRETTY_FUNCTION__ is a trace
T adder(T, Args ...) [with T = int; Args = int, int, int]      // expansion macro in gcc
T adder(T, Args ...) [with T = int; Args = int, int]
T adder(T, Args ...) [with T = int; Args = int]
T adder(T) [with T = int]
```

- We could also call as:

```
std::string s1 = "x", s2 = "aa", s3 = "bb", s4 = "yy";
std::string ssum = adder(s1, s2, s3, s4); // "x"+"aa"+"bb"+"yy" = "xaabbyy"
```

- `adder` will accept any number of arguments, and will compile properly as long as it can apply the `operator+` to them following template and overload resolution rules
- Variadic templates are like recursive code with a base case (`adder(T v)`) and a general case which *recurses* as in `adder(args...)`
- In `adder` - the first argument is peeled off the template parameter pack into type `T` (argument first). So with each call, the parameter pack shortens by one parameter to hit the base case

Variadic templates: Example: power_square

- Let us consider another function for practice:

```
#include <iostream>

template <typename T>T square(T t) { return t * t; } // A function that 'squares' a number
template <typename T> // Our base case just returns the value
double power_sum(T t) { cout << __PRETTY_FUNCTION__ << endl; return t; }
template <typename T, typename... Rest> // Our new recursive case
double power_sum(T t, Rest... rest) { cout << __PRETTY_FUNCTION__ << endl;
    return t + power_sum(square(rest)...);
}
int main() {
    int result = power_sum(2, 4, 6);
    // 2 + power_sum(square(rest)...);
    // 2 + power_sum(square(4), square(6));
    // 2 + (square(4) + power_sum(square(rest)...))
    // 2 + (square(4) + power_sum(square(square(6))));
    // 2 + (square(4) + (square(square(6))))
    std::cout << result;
}
double power_sum(T, Rest ...) [with T = int; Rest = int, int]
double power_sum(T, Rest ...) [with T = int; Rest = int]
double power_sum(T) [with T = int]
1314
```

Variadic templates: Example: count

- Consider:

```
#include <iostream>
template<typename... Types> // declare list struct
struct Count;               // walking template. Putting { } is optional

template<> struct Count<> { // recognize end of list
    const static int value = 0;
};

template<typename T, typename... Rest> // walk list
struct Count<T, Rest...> {
    const static int value = 1 + Count<Rest...>::value;
};

int main() {
    auto count1 = Count<int, double, char>::value; // count1 = 3
    auto count2 = Count<int>::value;               // count2 = 1
    auto count3 = Count<>::value;                  // count3 = 0
    std::cout << count1 << std::endl;
    std::cout << count2 << std::endl;
    std::cout << count3 << std::endl;
}
```

- A simple way to count the number of arguments

Local types as template arguments

- In C++03, local and unnamed types could not be used as template arguments. C++11 relaxes:

```
void f(vector<X>& v) {
    struct Less { bool operator()(const X& a, const X& b) { return a.v<b.v; } };
    sort(v.begin(), v.end(), Less()); // C++03: error: Less is local
                                      // C++11: okay
}
```

- In C++11, we also have the alternative of using a lambda expression:

```
void f(vector<X>& v) {
    sort(v.begin(), v.end(), [] (const X& a, const X& b)  return a.v < b.v; ); // C++11
}
```

- It is worth remembering that naming action can be quite useful for documentation and an encouragement to good design. Also, non-local (necessarily named) entities can be reused.
- C++11 also allows values of unnamed types to be used as template arguments:

```
template<typename T> void foo(T const& t) { }
enum X { x };
enum { y };
int main() {
    foo(x);       // C++03: okay; C++11: okay
    foo(y);       // C++03: error; C++11: okay
    enum Z { z };
    foo(z);       // C++03: error; C++11: okay
}
```

“>>” as Nested Template Closer

- >> now closes a nested template when possible:

```
std::vector<std::list<int>> vi1;  // fine in C++11, error in C++03
```

- The C++03 *extra space* approach remains valid:

```
std::vector<std::list<int> > vi2; // fine in C++11 and C++03
```

- For a shift operation, use parentheses:
  - That is, “>>” now treated like “>” during template parsing:

```
constexpr int n = ... ;                                  // n, m are compile-
constexpr int m = ... ;                                  // time constants
constexpr std::list<std::array<int, n  >> 2 >> L1; // error in C++03: 2 shifts
                                                         // error in C++11: 1st ">>"
                                                         // closes both templates
std::list<std::array<int, (n>>2) >> L2;                  // fine in C++11,
                                                         // error in C++03 (2 shifts)
```

Variable templates (C++14)

- A variable template may be introduced by a template declaration at namespace scope, where declaration declares a variable

```
#include <iostream>

template<typename T> T n = T(5); // variable template with default value: C++14

int main() { n<int> = 10; // instantiating variable template
    std::cout << n<int> << " ";    // instantiated value: 10
    std::cout << n<double> << " "; // default value: 5
}
```

- It can be constant too:

```
#include <iostream>
// variable template with constanr value
// math constant with precision dictated by actual type
template<typename T> constexpr T pi = T(3.14159265358979323846); // C++14
auto area_of_circle_with_radius = [](auto r) { return pi<decltype(r)> * r * r; }; // C++14
// template<class T> T area_of_circle_with_radius(T r) { return pi<T> * r * r; } // C++11

int main() { double r1 =  2.0; int r2 = 2;
    std::cout << area_of_circle_with_radius(r1) << std::endl; // for double: 12.5664
    std::cout << area_of_circle_with_radius(r2) << std::endl; // for int: 12
}
```

Module Summary

- Introduced several features in C++11 for non-class types and templates with examples
- Familiarized with important non-class types like enum class and fixed width integer
- Familiarized with important templates like variadic templates

Programming in Modern C++

Tutorial T11: Compatibility of C and C++: Part 1: Significant Features

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Tutorial Objectives

- We often say that "C is a subset of C++". It is far from truth. There are various intra-dialect incompatibilities in C (**C89**, **C99**, **C11**), in C++ (**C++03**, **C++11**, **C++14**, **C++17**, ...), and inter-dialect incompatibilities across languages. We need to understand these differences and their effect in the programs we write
- We take a look at the C/C++ communities and consider views of different sections of communities to understand why we need the compatibility - at least, the clear understanding for it
- We discuss the major compatibility issues between C and C++. To keep the discussion manageable, we primarily focus between **C99** and **C++11**
- We also discuss the workarounds to write more compatible code between C and C++

Tutorial Outline

1. Why is Compatibility of C and C++ important?
2. Compatibility of C and C++
   - `void*`
   - `const`
   - `enum`
   - ODR
   - `void` Param
   - Nested struct
   - VLA
   - FAM
   - `restrict`
3. Tutorial Summary

Why is Compatibility of C and C++ important?

Why is Compatibility of C and C++ important?

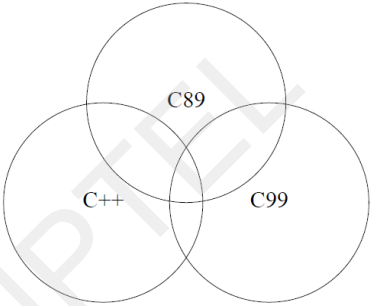
Source: The C/C++ Users Journal, Jul-Aug-Sep, 2002. Accessed 15-Sep-21
C and C++: Siblings, B. Stroustrup
C and C++: A Case for Compatibility, B. Stroustrup
C and C++: Case Studies in Compatibility, B. Stroustrup

Why is Compatibility of C and C++ important?

- The *C and C++ programming languages are closely related* but have *many significant differences*
- **There is no C/C++ language, but there is a C/C++ community**. Millions of programmers and organization who use C and/or C++ form the community comprising three major groups:
  - *Programmers who use C only*: Especially the embedded systems community. Many programmers working with C programs that never call a C++ library. However, most (?) C programmers occasionally use C++ directly and many rely on C++ libraries. Hence, the C programmer must be aware of C++ in the same way as a C++ programmer must be aware of C.
  - *Programmers who use C++ only*: Is it possible? Most programmers would need to call a C library. Hence, the programmer needs to understand the constructs in its header files - use of malloc() rather than new, the use of arrays rather than C++ standard library containers, and the absence of exception handling. So all C++ programmers are C programmers.
  - *Programmers who use both C and C++*:
- Compatibility maximizes the community of contributors. Each dialect and incompatibility limits the
  - market for vendors/suppliers/builders
  - set of libraries and tools for users - single product (IDE, compiler, analyzer, etc.) for both languages
  - set of collaborators (suitable employees, students, consultants, experts, etc.) for projects

Why is Compatibility of C and C++ important?

- Bjarne Stroustrup, the creator of C++, has suggested that *the incompatibilities between C and C++ should be reduced as much as possible in order to maximize interoperability between the two languages*
- Others argue that C and C++ are two different languages - *compatibility between them is useful but not vital*; and efforts to reduce incompatibility *should not hinder improvement of each language in isolation*
- **C99** "*endorse[d] the principle of maintaining the largest common subset*" between C and C++ "*while maintaining a distinction between them and allowing them to evolve separately*", and stated that the authors were "*content to let C++ be the big and ambitious language*"
- Several additions of **C99** are *not supported in the current C++ standard or conflicted with C++ features*, such as *variable-length arrays*, native *complex number* types and the *restrict* type qualifier
- On the other hand, **C99** *reduced some other incompatibilities compared with* **C89** by incorporating C++ features such as *// comments* and *mixed declarations and code*



Feature compatibility between C++98, C89, and C99. "There are features in all 7 areas" - C and C++: Siblings, B. Stroustrup, 2002

Compatibility of C and C++

Source: Accessed 15-Sep-21
C and C++: A Case for Compatibility, B. Stroustrup
C and C++: Case Studies in Compatibility, B. Stroustrup
Compatibility of C and C++, HandWiki
Compatibility of C and C++, Wikipedia
Annex C.1 of the ISO C++ standard

Compatibility of C and C++

- *C is not a subset of C++, and nontrivial C programs will not compile as C++ code without change*
- *Likewise, C++ introduces many features that are not available in C and in practice almost all code written in C++ is not conforming C code*
- Here, we focus on differences that cause **conforming C code to be ill-formed C++ code**, or to be **conforming / well-formed in both languages but to behave differently in C and C++**
- We take following approach for the discussions:
  - To explain the compatibility, incompatibility and work-around in an understandable way, we write the same code in `main.c` and `main.cpp` and compile with `gcc` to get the language specific behavior
  - We also use dialect specific `-std` flags wherever relevant. Most comparisons are done with respect to **C99** and **C++11**
  - We present the compiler messages and / or output to elucidate the effects
  - We present a summary of the compatibility issues at the end in comparative tabular form

Compatibility of C and C++: `void*`

- One commonly encountered difference is C being more weakly-typed regarding pointers
- **Specifically, C allows a `void*` pointer to be assigned to any pointer type without a cast, while C++ does not**
- This idiom appears often in C code using `malloc` memory allocation, or in the passing of context pointers to the `POSIX pthreads API`, and other frameworks involving `callbacks`
- For example, the following is valid in C but not C++:

```
void *ptr;
/* Implicit conversion from void* to int* */
int *i = ptr;
```

or similarly:

```
int *j = malloc(5 * sizeof *j); /* Implicit conversion from void* to int* */
```

In order to make the code compile as both C and C++, one must use an explicit cast, as follows (with some caveats in both languages)

```
void *ptr;
int *i = (int *)ptr;
int *j = (int *)malloc(5 * sizeof *j);
```

Compatibility of C and C++: const

- C++ is also more strict than C about pointer assignments that discard a `const` qualifier. For example, assigning a `const int*` value to an `int*` variable:

```
int main() { const int* p = 0;
    int* q = p; // const qualifier being discarded
}
```

In C++, this is invalid and generates a compiler error (unless an explicit typecast is used), while in C this is allowed (although many compilers emit a warning)

```
$ gcc main.cpp main.c

main.cpp:3:14:  error: invalid conversion from 'const int*' to 'int*' [-fpermissive]
     int* q = p;
              ^
main.c:3:14:  warning: initialization discards 'const' qualifier from pointer target type
                       [-Wdiscarded-qualifiers]
     int* q = p;
              ^
```

- In C++ a const variable must be initialized; in C this is not necessary. For

```
int main() { const int i = 5;
    const int j; // const variable not initialized
}
$ gcc main.cpp main.c

main.cpp:12:15:  error: uninitialized const 'j' [-fpermissive]
     const int j;
               ^
```

Compatibility of C and C++: string.h and enum

- C++ changes some C standard library functions to add additional overloaded functions with `const` type qualifiers, for example, consider `strchr()` function in `string.h` in C and `cstring` in C++

```
// string.h
char *strchr(const char *str, int character)
// cstring
const char *strchr(const char * str, int character);
char *strchr (char * str, int character);
```

  So when a C file is compiled with C++ compiler different calls to `strchr()` may bind to different overloads in C++
- C++ is also more strict in conversions to `enum`s: `int`s cannot be implicitly converted to `enum`s as in C.

```
enum week { Mon, Tue, Wed, Thur, Fri, Sat, Sun };
int main() { enum week day;
    int dayindex = 2;
    day = dayindex;
}

$ gcc main.c main.cpp

main.cpp:23:11:  error: invalid conversion from 'int' to 'week' [-fpermissive]
     day = dayindex;
           ^~~~~~~~
```

- Also, Enumeration constants (`enum` enumerators) are always of type `int` in C, whereas they are distinct types in C++ and may have a size different from that of `int`

Compatibility of C and C++: One Definition Rule (ODR)

- C allows for multiple tentative definitions of a single global variable in a *single translation unit*, which is disallowed as an *One Definition Rule (ODR)* violation in C++

```
int N;
int N = 10;

$ gcc main.c main.cpp
main.cpp:46:5:  error: redefinition of 'int N'
 int N = 10;
     ^
main.cpp:45:5:  note: 'int N' previously declared here
 int N;
     ^
```

- C allows declaring a new type with the same name as an existing `struct`, `union` or `enum` which is not allowed in C++, as in C `struct`, `union` or `enum` types must be indicated as such whenever the type is referenced whereas in C++ all declarations of such types carry the `typedef` implicitly

```
enum BOOL { FALSE, TRUE };
typedef struct _BOOL { int b; } BOOL;

$ gcc main.c main.cpp
main.cpp:53:33:  error: conflicting declaration 'typedef struct _BOOL BOOL'
 typedef struct _BOOL { int b; } BOOL;
                                 ^~~~
main.cpp:52:6:  note: previous declaration as 'enum BOOL'
 enum BOOL { FALSE, TRUE };
      ~~~~
```

Compatibility of C and C++: void Parameter

- In C, a function prototype without parameters, for example, `int foo();`, implies that the parameters are unspecified. Therefore, it is legal to call such a function with one or more arguments, like `foo(0)`
- In contrast, in C++ a function prototype without arguments means that the function takes no arguments, and calling such a function with arguments is ill-formed
- **In C, declare a function taking no argument by using `void`, as in `int foo(void);`, which is also valid in C++. Empty function prototypes are a deprecated feature in C99 (as they were in C89)**

```
int foo(); int bar(void);
int main() { foo(0); bar(0); }
$ gcc main.c main.cpp
main.c:42:22:  error: too many arguments to function 'bar'
 int main() { foo(0); bar(0); }
                      ~~~
main.c:41:16:  note: declared here: int foo(); int bar(void);
                                          ~~~

main.cpp:59:19:  error: too many arguments to function 'int foo()'
 int main() { foo(0); bar(0); }
                   ^
main.cpp:58:5:  note: declared here: int foo(); int bar(void);
                                        ~~~

main.cpp:59:27:  error: too many arguments to function 'int bar()'
 int main() { foo(0); bar(0); }
                           ^
main.cpp:58:16:  note: declared here: int foo(); int bar(void);
                 ~~~
```

Compatibility of C and C++: Nested struct

- In both C and C++, one can define nested `struct` types, but the scope is interpreted differently
  - In C++, a nested `struct` is defined only within the scope / namespace of the outer `struct`
  - In C the inner `struct` is also defined outside the outer `struct`

```
struct Outer {
    int o;
    struct Inner {
        int i;
    };
};

struct Outer O1;        // Okay in C and C++

#ifndef __cplusplus
struct Inner I1;        // Okay only in C
#endif
```

Compatibility of C and C++: Variable Length Array (VLA)

- **Variable Length Arrays (VLA)** is a feature where we can allocate an auto array (on stack) of variable size. C supports variable sized arrays from C99 standard
- But, in C++ standard (till **C++11**) there was no concept of VLA. According to the **C++11** standard, array size is a constant-expression. In **C++14** mentions array size as a simple expression (not constant-expression)

```
#ifndef __cplusplus
#include <stdio.h>
// VLA in function prototype
int add(int x, int a[*]);
// int add(int x, int a[]); also works
#else
#include <cstdio>
using namespace std;
// Unspecified size in function prototype
int add(int x, int a[]);
#endif
```

```
int set_and_add(int n) { int vals[n]; // Variable Length Array
    printf("%d ", sizeof(vals));      // Runtime sizeof
    for (int i = 0; i < n; ++i) vals[i] = i;
    return add(n, vals);
    // vals is declared as an automatic variable
    // its lifetime ends when add() returns
}
int main() { int n = 5;
    printf("Result = %d", set_and_add(n));
}
int add(int n, int a[]) { int sum = 0;
    for (int i = 0; i < n; ++i) sum += a[i];
    return sum;
}
```

- The above code uses VLA (`int vals[n]`)) in function `set_and_add`. So any size (bounded by a compiler-specified maximum) can be passed to it
- VLA may lead to possibly non-compile time `sizeof` operator

Compatibility of C and C++: Flexible Array Member (FAM)

- The last member of a **C99** structure type with more than one member may be a **Flexible Array Member (FAM)**, which takes the syntactic form of an array with unspecified length. This serves a purpose similar to variable-length arrays
- VLAs cannot appear in type definitions, but has defined size (at runtime)
- FAMs have no defined size, but can appear in type definitions
- **ISO C++ has no such feature**
- Here is an example of a FAM

```
struct vectord {
    short len;      // there must be at least one other data member
    double arr[];   // the flexible array member must be last
                    // The compiler may reserve extra padding space here,
                    // like it can between struct members
};
```

- Typically, such structures serve as the header in a larger, variable memory allocation

```
struct vectord *vector = malloc(...);
vector->len = ...;
for (int i = 0; i < vector->len; i++)
    vector->arr[i] = ...;  // transparently uses the right type (double)
```

Compatibility of C and C++: `restrict`

- `restrict` keyword is mainly used in pointer declarations as a type qualifier for pointers
- It adds no functionality - only informs the compiler about an optimization
- When we use `restrict` with a pointer `ptr`, it tells the compiler that `ptr` *is the only way to access the object pointed by it*, in other words, *there is no other pointer pointing to the same object*. That is, `restrict` keyword *specifies that a particular pointer argument does not alias any other* and the *compiler does not need to add any additional checks*
- If a programmer uses `restrict` keyword and violate the above condition, the behavior is undefined
- `restrict` is supported from C99. **It not supported by ISO C++**

```
#include <stdio.h>
// The purpose of restrict is to show only syntax. It does not change anything in output (or logic)
// It is just a way for programmer to tell compiler about an optimization
void use(int* a, int* b, int* restrict c) {
    *a += *c;
    // Since c is restrict, compiler will not reload value at address c in its assembly code
    // Therefore generated assembly code is optimized
    *b += *c;
}
int main(void) { int a = 50, b = 60, c = 70;
    use(&a, &b, &c);
    printf("%d %d %d", a, b, c);
}
```

Source: restrict keyword in C and How to Use the restrict Qualifier in C Accessed 15-Sep-21

Tutorial Summary

- We have understood why C and C++ incompatible across dialects in spite of C++ being an intended super-set of C
- We studied specific incompatibilities over nearly two dozen features
- We discussed some workarounds to write more compatible code between C and C++